LLMs for Social Scientists

POL42560 AI and Large Language Models

Lorcan McLaren

February 2026

Table of contents

Welcome **7**
Overview . . . 7
Learning Outcomes . . . 7
Prerequisites . . . 8
Course Details . . . 8
License . . . 8

I Course Information **9**

Syllabus **10**
Course Structure . . . 10
Schedule at a Glance . . . 10
Textbooks and Readings . . . 11
Approachable . . . 11
Deeper Dive . . . 11

Assessment **12**
Components . . . 12
Lab Participation . . . 12
Group Presentation . . . 12
Group Project . . . 13
Submission Format . . . 13
Research Paper Structure . . . 13

Software **15**
Development Environment . . . 15
Package Management . . . 15
Useful Python Packages . . . 15

Policies **16**
AI Use . . . 16
Documenting AI Use . . . 16
Suggested AI Tools . . . 17
Plagiarism . . . 17
Late Submission . . . 17

Office Hours . . . 17
Syllabus Modification Rights . . . 17

II Weekly Content **18**

1 Introduction **19**
1.1 Lecture . . . 19
1.1.1 Module overview . . . 19
1.1.2 A brief overview of AI . . . 20
1.1.3 Large language models . . . 21
1.1.4 Why should political (or social) scientists care? . . . 21
1.2 Lab . . . 21
1.3 Readings . . . 22

2 Text-as-Data **23**
2.1 Lecture . . . 23
2.1.1 Content analysis . . . 23
2.1.2 Quantitative text analysis . . . 24
2.1.3 Process . . . 24
2.1.4 Applications . . . 25
2.2 Lab . . . 26
2.3 Readings . . . 27

3 Embeddings **28**
3.1 Lecture . . . 28
3.1.1 From bag-of-words to embeddings . . . 28
3.1.2 What are embeddings? . . . 29
3.1.3 Geometric properties of embeddings . . . 29
3.1.4 Training word embeddings . . . 30
3.1.5 Bias in word embeddings . . . 30
3.1.6 Applications of embeddings . . . 30
3.1.7 Sentence, paragraph, and document embeddings . . . 31
3.1.8 Embeddings beyond text . . . 32
3.2 Lab . . . 32
3.3 Readings . . . 32

4 Transformers **33**
4.1 Lecture . . . 33
4.1.1 The transformer revolution . . . 33
4.1.2 Attention . . . 34
4.1.3 Embeddings in transformers . . . 34
4.1.4 Transformers and LLMs . . . 35

4.1.5 Model architectures . . . 35
4.1.6 Pre-training and fine-tuning . . . 36
4.1.7 Zero-shot classification . . . 36
4.1.8 Applications in political science . . . 36
4.1.9 Considerations . . . 37
4.2 Lab . . . 37
4.3 Readings . . . 37

5 Validation and Performance Measurement 38
5.1 Lecture . . . 38
5.1.1 Validation in LLM research . . . 39
5.1.2 Ground truth . . . 39
5.1.3 Confusion matrices . . . 39
5.1.4 Performance metrics . . . 40
5.1.5 Multiclass averaging methods . . . 41
5.1.6 Standard benchmarks vs. domain validation . . . 42
5.1.7 Task-specific evaluation metrics . . . 42
5.1.8 Crowdcoding . . . 42
5.1.9 Codebooks and annotation . . . 43
5.1.10 Intercoder reliability . . . 43
5.1.11 Human evaluation challenges . . . 44
5.1.12 Open science and reproducibility . . . 44
5.2 Lab . . . 46
5.3 Readings . . . 46

6 Generative Language Models I 47
6.1 Lecture . . . 47
6.1.1 The text analysis research pipeline . . . 47
6.1.2 Generative language models . . . 48
6.1.3 Generative models in political science . . . 50
6.2 Lab . . . 51
6.3 Readings . . . 52

7 Generative Language Models II 53
7.1 Lecture . . . 53
7.1.1 LLMs vs. human annotators . . . 53
7.1.2 Prompt engineering . . . 53
7.1.3 Hyperparameters . . . 56
7.1.4 Conceptual clarity . . . 57
7.2 Lab . . . 57
7.3 Readings . . . 57

8 Bias in LLMs **58**
8.1 Lecture . . . 58
8.1.1 Defining bias in AI systems . . . 58
8.1.2 Examples of algorithmic bias . . . 59
8.1.3 Sources of bias in LLMs . . . 60
8.1.4 Measuring bias in LLMs . . . 61
8.1.5 Political leanings of LLMs . . . 61
8.1.6 Bias mitigation techniques . . . 61
8.1.7 Challenges in bias mitigation . . . 62
8.1.8 Unintended consequences and backlash . . . 62
8.1.9 Open questions . . . 63
8.1.10 Frontiers . . . 63
8.2 Lab . . . 63
8.3 Readings . . . 63

9 Ethical Use of LLMs? **64**
9.1 Lecture . . . 64
9.1.1 Bias risks . . . 64
9.1.2 Data privacy concerns . . . 65
9.1.3 False or misleading information . . . 65
9.1.4 Malicious use potential . . . 66
9.1.5 Implications of overreliance . . . 66
9.1.6 Environmental costs . . . 67
9.1.7 Negative economic externalities . . . 68
9.1.8 Output ownership and responsibility . . . 68
9.1.9 Open questions . . . 69
9.2 Lab . . . 69
9.3 Readings . . . 69

10 Interpretability and Explainability **70**
10.1 Lecture . . . 70
10.1.1 The challenge of interpretation . . . 70
10.1.2 Why interpret or explain? . . . 71
10.1.3 Interpretability vs. explainability . . . 71
10.1.4 Two paradigms of LLM explainability . . . 72
10.1.5 Encoder-only models: the fine-tuning paradigm . . . 72
10.1.6 Decoder-only models: the prompting paradigm . . . 73
10.1.7 Evaluating explanations . . . 74
10.2 Lab . . . 75
10.3 Readings . . . 75

11 Beyond Text **76**
11.1 Lecture . . . 76
11.1.1 Audio . . . 76
11.1.2 Image . . . 78
11.1.3 Video . . . 79
11.1.4 Multimodality . . . 79
11.1.5 Frontier applications . . . 80
11.2 Lab . . . 80
11.3 Readings . . . 80

Appendices **81**

A Datasets **81**
A.1 Party Manifestos and Political Programs . . . 81
A.2 Parliamentary Speeches . . . 81
A.3 Political Advertising . . . 81
A.4 News and Legal Documents . . . 81
A.5 Collections . . . 82

References **83**

Welcome

This is the course website for **POL42560 AI and Large Language Models**, taught at University College Dublin in the Spring Trimester 2026.

Overview

Large language models (LLMs), such as those behind tools like ChatGPT, have garnered significant attention for their ability to generate human-like text, sparking both enthusiasm and debate about their implications for various fields. For social scientists, LLMs offer a potentially useful approach to processing and interpreting vast amounts of data, enhancing our ability to study complex societal issues.

This course offers an interdisciplinary approach to understanding and applying LLMs in social and political science, with a focus on text analysis. It combines theoretical foundations with practical, hands-on experience in applying LLMs to address substantive social science questions. Students will explore the capabilities and limitations of LLMs and engage critically with issues such as bias, environmental impact, misinformation, and intellectual property rights. They will also become familiar with essential research practices including documentation, reproducibility, and validation.

Learning Outcomes

By the end of the course, students will be able to:

1. Understand the fundamentals of large language models and their applications in social sciences.
2. Implement LLMs for various tasks relevant to political and social science research using Python.
3. Critically evaluate the ethical and societal implications of AI technologies, particularly LLMs, in the context of social science.
4. Apply best practices in research documentation, reproducibility, and validation when using AI tools.
5. Engage in informed discussions about the impact of AI on society, including issues of bias, sustainability, and concentration of power.

Prerequisites

This course is designed for students with or without experience in Python, but a willingness to learn quickly and engage with technical content is essential. Students without prior programming experience would benefit from taking **POL42340 Programming for Social Scientists** alongside this course to familiarise themselves with Python fundamentals.

POL42050 Quantitative Text Analysis provides a broader overview of text-as-data approaches, complementary to the content of this course.

Course Details

Time	Fri, 11:00 – 12:50
Format	In-person lectures and labs
Credits	10, Level 4
Instructor	Lorcan McLaren
Email	lorcan.mclaren@ucdconnect.ie
Office Hours	Wed, 15:00 – 16:00 (sign up)

License

This work is licensed under a Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License.

Part I

Course Information

Syllabus

Course Structure

Each class involves a lecture introducing the topic, followed by a hands-on lab session where we work on exercises related to the lecture content. Students are expected to complete the readings before class each week, and to submit their lab work via Brightspace.

See the Weekly Content chapters for detailed content for each session.

Schedule at a Glance

Week	Topic	Lab
1	Introduction	Setting up a Python development environment
2	Text-as-Data	Dealing with textual data in Python
3	Embeddings	Measuring text similarity using embeddings
4	Transformers	Fine tuning transformers for text classification
5	Validation and Performance Measurement	Measuring model performance with `scikit-learn`
6	Generative Language Models I	Introduction to HuggingFace `transformers`
7	Generative Language Models II	Introduction to `langchain`
–	*Study Break*	

Week	Topic	Lab
8	Interpretability, Explainability and Bias	Applying interpretability tools; visualising bias
–	*Good Friday*	
10	Ethical Use of LLMs?	Measuring environmental costs of LLM use
11	Beyond Text	Working with images, audio and video in Python
12	Project Presentations	

Textbooks and Readings

All mandatory readings are indicated in the weekly chapters and are open access or available through the UCD Library. Readings should be completed ahead of class each week.

Approachable

- Alammar, J. and Grootendorst, M. (2024) *Hands-On Large Language Models: Language Understanding and Generation*. 1st edition. O'Reilly Media.
- Grimmer, J., Roberts, M.E. and Stewart, B.M. (2022) *Text as Data: A New Framework for Machine Learning and the Social Sciences*. Princeton University Press.

Deeper Dive

- Raschka, S. (2024) *Build a Large Language Model (From Scratch)*. Manning.
- Kamath, U. et al. (2024) *Large Language Models: A Deep Dive: Bridging Theory and Practice*. Springer.

Assessment

Components

Component	Deadline	Grade %
Lab participation	Continuous	20
Group presentations	April 24	20
Group project	May 1	60

Lab Participation

Each lab session includes a Jupyter notebook that must be submitted through Brightspace for participation credit. While your solutions don't need to be perfect, they should demonstrate a genuine attempt at completing all exercises. Include clear comments throughout your code to show your understanding of the concepts.

You should **execute all code cells prior to saving**, so that the output for each cell is visible in the submitted version.

You will have **one week to submit the notebook** from each lab session, up to the start of the subsequent class (e.g., the notebook for Week 2 must be submitted by 08:59 on Friday in Week 3). You may miss up to one submission without prior approval. Late submissions will not be accepted unless an extension has been granted, as exercise solutions may be discussed in class.

Group Presentation

Each group must deliver a presentation lasting **no more than 12 minutes**, using either LaTeX Beamer, PowerPoint or Google Slides. Submit your slides on Brightspace by **23:59 on Wednesday, April 22**. Groups should divide the presentation between 2–3 speakers, with non-presenting members taking responsibility for answering questions from peers and the instructor during the Q&A session.

You should prepare together by practising the presentation and anticipating potential questions. The same grade will be awarded to all group members, unless one or more members fail to participate in both the presentation and Q&A.

Your presentation should cover:

1. Research question and importance
2. Theoretical expectations and main hypothesis
3. Data description
4. Methodology for hypothesis testing
5. Initial results (if available)
6. Challenges encountered and potential risks for next steps (if applicable)

Group Project

Each group must submit a **4,000–5,000 word** research paper (**excluding references and appendices**) as well as the code used to conduct the analysis. All students in a group will be awarded the same grade, but you may be asked to indicate who contributed to which parts of the project.

You have free choice in selecting your research question, but you must justify why it is relevant and important for understanding or addressing a meaningful social science question. You may either collect original data appropriate for your research question or identify and use existing datasets (see Appendix A), clearly explaining your data source selection. Your analysis must incorporate **at least one** of the methods covered in this course – embeddings, transformers, or generative models – in a way that meaningfully advances your research objectives.

Submission Format

1. A research paper in PDF format. The paper must be written in LaTeX using the collaborative Overleaf document that will be created for each group.
2. A GitHub repository containing all code needed to reproduce your analysis.

Research Paper Structure

The paper should contain the following sections:

1. **Introduction** – Research question/puzzle, significance, main hypothesis
2. **Theory** – Theoretical framework, prior findings on DV-IV relationship, expected effects
3. **Data & Methods** – Dataset overview, unit of analysis, sample size & missing data, analysis approach

4. **Results** – Empirical findings, statistical tests, key patterns
5. **Conclusion** – Link to research question, main insights, limitations, broader implications

Software

All code for this course should be written in **Python**. For those who haven't worked with this language previously, POL42340 Programming for Social Scientists runs in parallel to this course and provides a comprehensive introduction to Python.

Development Environment

We will use **VS Code** to write and execute code locally. For some lab sessions and the group project, where your personal device may not be powerful enough or where long execution times are expected, **Google Colab** should be used. Colab provides limited free access to T4 GPUs, which will enable you to run code that cannot be executed locally without a GPU.

Package Management

We will use `venv` and `pip` to manage our Python environment and install packages (though feel free to use Anaconda if you are more familiar with this platform). I strongly encourage the use of **Git** for version control, particularly when working on your group project; however, we will not cover this extensively in class.

Useful Python Packages

The following packages are particularly useful for dealing with textual data:

Package	Description
`spaCy`	Industrial-strength NLP library
`NLTK`	Classic NLP toolkit
`Gensim`	Topic modelling and word embeddings
`sentence-transformers`	Sentence and document embeddings
`transformers`	HuggingFace model hub and pipelines
`langchain`	Framework for building LLM applications
`scikit-learn`	Machine learning and evaluation metrics

Policies

AI Use

I encourage the use of generative AI tools when completing the assignments for this module, but all work relying on AI-generated content must adhere to the highest academic standards. Users of this technology must be aware of what it can and, more importantly, what it cannot do well. It is crucial for you to exercise judgement when evaluating the quality and reliability of content generated through AI platforms.

AI is not a panacea for all writing challenges; it will not automatically generate a flawless, logically coherent, and factually correct assignment. Instead, use AI as a tool to tackle specific issues such as brainstorming and idea formation, literature discovery, and text drafting. View your preferred AI platform(s) as useful but imperfect tools that can offer inspiration, new perspectives, and supplementary areas for research for your own work. In-depth research on your part remains essential to ensure coherent, factual, and scientifically informed perspectives in your assignment. Always cross-reference the information AI offers against other independent and reliable sources.

AI use must be in line with UCD's policies on academic integrity.

Documenting AI Use

You are expected to provide an account of the tools used and how they were used in a **mandatory appendix** to your group project. This appendix will be assessed as part of each assignment, with grade points awarded for effective communication of the methods used to generate content.

For each instance where a generative AI tool is used, you need to provide:

1. An in-text citation or footnote
2. A bibliographic reference to the tool used and the date of access
3. An entry in the mandatory AI appendix detailing how the tool was used

For code, while I do not expect you to keep a log of all prompts used, I do expect you to add comments indicating where code has fully or partially been generated by a tool such as ChatGPT or GitHub Copilot.

Suggested AI Tools

- **ChatGPT** – good at writing code
- **Claude** – better than ChatGPT at writing text
- **Gemini** – fast and useful integrations
- **Perplexity** – great for finding relevant web pages and documents
- **HuggingChat** – open source, flexible, and free

Plagiarism

Academic integrity is one of the core values of the UCD Education Strategy. Plagiarism is the inclusion, in any form of assessment, of material without due acknowledgement of its original source. See UCD's Academic Integrity Policy for full details.

Late Submission

Lab notebooks submitted late will not be accepted unless an extension has been granted. For group projects submitted past the due date:

- Coursework received within **5 working days** after the due date will have the grade reduced by **one grade point**.
- Coursework received between **5 and 10 working days** after the due date will have the grade reduced by **two grade points**.
- Coursework received **more than 10 working days** after the due date will not be accepted.

Office Hours

Weekly office hours during the teaching term: Wednesdays, 15:00–16:00. Book a slot via Calendly if you wish to attend, either in person or online.

Syllabus Modification Rights

I reserve the right to reasonably alter the elements of the syllabus at any time to keep pace with the course schedule. If adjustments are made, the revised syllabus will be uploaded to Brightspace.

Part II

Weekly Content

1 Introduction

Week 1

Lecture: Module overview and organisation
Lab: Setting up a Python development environment

1.1 Lecture

1.1.1 Module overview

This module takes a utilitarian approach to AI and large language models (LLMs): how can we maximise the utility of LLMs and generative AI for us as political and social scientists, while minimising the harms? The module is research-led and driven by readings from current scholarship, with an emphasis on active and collaborative learning and transparent and reproducible practices.

Companion modules

POL42340 Programming for Social Scientists runs in parallel and provides an introduction to Python that is highly useful if you do not have prior Python experience. **POL42050 Quantitative Text Analysis** provides a broader overview of text-as-data approaches that is complementary to this course.

The primary focus of the module is on LLMs and their applications in social science research, particularly using text-as-data. We will not focus extensively on other AI technologies such as speech-to-text, text-to-speech, or music/image/audio generation, as these are less practically relevant to social science research at present. Image recognition and multimodality may be touched upon, but these remain new tools for social science and are not yet in widespread use in scholarship. The module also considers the implications of these technologies for politics and society.

1.1.2 A brief overview of AI

1.1.2.1 What is artificial intelligence?

Artificial intelligence can be defined as a rational agent: an automated system that does the "right thing", based on its objective, given its environment (Russell and Norvig 2020). In the past, attempts at creating AI were dominated by a rules-based approach (or "expert systems"), in which knowledge and logic are encoded as a set of rules and a system makes decisions based on these. More recently, machine learning approaches have come to predominate. While machine learning and neural networks have existed since the 1960s and saw key innovations in the 1980s, it was increases in computational power and the rise of big data in the 2000s that led to a shift away from rules-based systems. Machine learning systems are able to handle uncertainty by learning patterns from a large training dataset in a way that often surpasses the performance of more brittle, handwritten rules.

AI, machine learning, and deep learning are nested concepts. AI is the broadest category, machine learning is a subset of AI, and deep learning is a subset of machine learning.

1.1.2.2 Types of machine learning

Machine learning can be divided into three broad categories:

- **Supervised learning** is when we have a training dataset for which we know the outcomes or "labels", and use this to train a model for unseen data. *Classification* is when we wish to train a model to place observations into discrete categories (e.g. categorising images as containing cats or dogs). *Regression* models have continuous rather than discrete outcomes (e.g. predicting property prices).
- **Unsupervised learning** is when the outcomes or "labels" are not known for our dataset. For example, we might group similar images into clusters, without explicitly knowing how these clusters are defined. We might later find that these clusters correspond to discrete categories (e.g. animals), but we do not know this beforehand.
- **Reinforcement learning** is where a model is trained to make a series of decisions sequentially, such that each outcome is dependent on the previous, rather than being independent as in supervised learning. The goal is to find the optimal set of actions that maximise expected rewards (e.g. AI models that learn to play games).

1.1.2.3 Neural networks and deep learning

One well-known family of machine learning architecture is the neural network. These are networks of nodes (or "neurons") connected to each other by weighted edges. Each node is an

individual mathematical function. Input data passes through layers to generate predictions or classifications. Neural networks can be used for both supervised and unsupervised learning.

Deep learning refers to the use of "deep" neural networks with one or more hidden layers. Each layer extracts increasingly complex features from the data. Deep neural networks learn useful feature representations automatically, allowing them to handle various types of data from text to image to audio.

1.1.3 Large language models

LLMs are fundamentally prediction machines — they predict the next token (word or part of word) based on the previous context. Through this simple objective and massive training data, they appear to learn complex language understanding. Key characteristics include:

- Training on internet-scale datasets
- The ability to handle multiple tasks without task-specific training
- Learning patterns of language and knowledge during pre-training

The hype around LLMs stems from their apparent ability to perform tasks previously thought to require human intelligence, including the emergence of "reasoning" capabilities through chain-of-thought prompting, the ability to follow instructions and adapt to new tasks, and integration into research workflows. However, it is important to maintain perspective: despite the module's name, we should avoid the use of "artificial intelligence" in favour of more specific terminology. LLMs are fundamentally statistical pattern matching systems, with significant limitations. They do not "think" or "reason" — they predict patterns. They can be confidently wrong and require careful human oversight.

1.1.4 Why should political (or social) scientists care?

Political scientists need to engage with LLMs in order to understand:

1. **Policy implications and governance challenges** posed by these technologies
2. **How to use these tools responsibly in research**
3. **Their impact on information environments**

1.2 Lab

Setting up a Python development environment.

1.3 Readings

- Grossmann et al. (2023)
- Ziems et al. (2023)

2 Text-as-Data

> **Week 2**
>
> **Lecture:** Tasks in text-as-data (scaling, topic modelling, classification, clustering, keyword extraction, NER, sentiment analysis)
> **Lab:** Dealing with textual data in Python

2.1 Lecture

Text is one of the main ways we study politics. Most features of politics that we might be interested in have been or can be encoded as textual data in some way: opinion, motivation, actions, framing, outcomes. Opinion polling and experiments are distinct, but almost all other research approaches, whether qualitative or quantitative, attempt to analyse text in one form or another.

2.1.1 Content analysis

Content analysis is used to understand the (self-)presentation of political actors and policy content. It uses human judgement not to make sense of each text directly, but instead to apply a scheme to convert the text into data by recording labels or ratings for each unit of text. Text reflects how politicians seek to project an image of themselves, their actions, and others. For example, we might review a series of speeches from MPs on budgetary measures and annotate or "code" them as fiscally right-wing or left-wing. Content analysis can also be applied to legal texts and policy documents.

2.1.1.1 Human vs. automated content analysis

Humans are flexible and able to handle nuance and ambiguity, but are vulnerable to subjectivity and performance variability over time. Even trained expert coders — the "gold standard" — are not failure-proof. The Manifesto Project, a major dataset of political texts annotated at the sentence-level with labels for policy area, has been found to have levels of inter-rater agreement and reliability so low that, had the coders been oncologists, their levels of tumour

misdiagnosis would have been medically and financially catastrophic (Mikhaylov, Laver, and Benoit 2012). Automated methods are more consistent, cost- and time-efficient, but they may be consistently wrong. Humans are still typically used as the point of reference ("ground truth") against which we compare other approaches.

2.1.2 Quantitative text analysis

In quantitative text analysis, we convert each document into some kind of numeric representation which we can work with more easily. The document-feature matrix (DFM) is one widely used representation that records the frequency of each feature (typically a word) across each document. From even a simple DFM, we can make initial observations — for instance, comparing how frequently different US presidents mention terms like "economy", "crime", or "climate" in their State of the Union addresses.

Four key principles guide quantitative text analysis (Grimmer and Stewart 2013):

1. All quantitative models of language are wrong — but some are useful.
2. Quantitative methods for text amplify resources and augment humans.
3. There is no globally best method for automated text analysis.
4. Validate, validate, validate.

2.1.3 Process

The typical text-as-data workflow follows seven steps (Benoit 2020):

1. **Selecting texts and defining the corpus.** Identify the corpus of texts relevant to the research question. Texts are generally distinguished from one another by attributes relating to the author, speaker, time, or topic. Be aware of selection issues and data gaps (e.g. coalition manifestos tend to disappear after a coalition collapses, leading to a bias in the dataset towards more stable coalitions).

2. **Converting texts into a common electronic format.** This involves converting PDFs and image formats to text via OCR, standardising file types (HTML, XML, JSON, Word, Excel, TXT, TSV, CSV), handling character encoding issues (e.g. accents or special characters), and text cleaning (whitespace, page numbers). This step is often glossed over, but getting or creating a clean, organised dataset can be one of the most time-consuming components of any text-as-data project.

3. **Defining documents and choosing the unit of analysis.** Documents could be individual social media posts, newspaper articles, parliamentary speeches, and so on. Consider what level of granularity is needed to answer the research question. Texts are often analysed at the sentence level and then aggregated afterwards.

4. **Defining and refining features.** A "type" refers to the abstract category of a word or concept, while a "token" is a specific instance of that type as it appears in text. Preprocessing steps include tokenisation, casefolding, stemming, lemmatisation, stopword removal, and the identification of multi-word expressions (n-grams). Tokenisation can be more challenging in languages such as Chinese (with no spaces between words) or in morphologically rich languages like Finnish or Turkish.
5. **Converting textual features into a quantitative matrix.** This yields an N×M matrix, where N is the number of documents and M is the number of features. Values typically start as raw frequency counts but may be re-weighted via normalisation or tf-idf (a measure of importance of a word to a document in a collection, adjusted for the fact that some words appear more frequently in general). Trimming may be applied to overcome sparsity by filtering based on term frequency.
6. **Analysing the matrix data using an appropriate statistical procedure.**
7. **Interpreting and reporting the results.**

2.1.4 Applications

2.1.4.1 Characterising text

Keyword extraction and analysis involves identifying the most frequent or distinctive terms in a corpus. For example, keyword datasets have been used to track how European political parties on Facebook discuss the migration crisis, with variation across countries and ideological groups (Caravaca, Cuevas, and Cuevas 2025).

Named entity recognition (NER) identifies people, organisations, and other entities mentioned in text. Jaros and Pan (2018) used NER in Chinese official provincial newspapers to recognise all people and organisations named in Party newspapers, using the frequency of mentions as a measure of power. They found that Xi Jinping was much more frequently mentioned in Party newspapers than his predecessor.

Sentiment analysis measures the emotional tone of text. Rheault et al. (2016) tracked the emotional polarity of government and opposition speeches in Britain from 1946 to 2013.

Readability analysis assesses how easy texts are to understand. Benoit, Munger, and Spirling (2019) examined the readability of US State of the Union addresses over time, finding a trend towards simpler language.

2.1.4.2 Classifying into known categories

Dictionary analysis uses predefined lists of words to classify text. Dictionaries are interpretable but brittle: they do not generalise well to other datasets and are vulnerable to polysemy (e.g. the word "kind" can mean both "type" and "generous").

Supervised machine learning is less interpretable but more flexible. A model is trained on labelled examples and then applied to classify unseen documents. For example, Müller and Proksch (2024) used DistilBERT to measure party-level nostalgia across European democracies.

2.1.4.3 Discovering categories

Topic modelling is an unsupervised method in which clusters of co-occurring words represent "topics" and documents contain them in relative proportion. Estimated topics are unlabelled, so a human must assign labels by interpreting the content of the words most highly associated with each topic. Choosing the correct number of topics involves balancing statistical measures and interpretability.

2.1.4.4 Measuring latent features

Supervised scaling methods such as Wordscores (Laver, Benoit, and Garry 2003) require some reference texts for which a score (e.g. left–right position) is known, from which the model learns to score new texts.

Unsupervised scaling methods such as Wordfish (Slapin and Proksch 2008) run without reference texts. It is up to the researcher to interpret what the estimated dimensions actually represent. A risk is that the model might capture thematic differences (e.g. talking about the economy vs. foreign policy) rather than ideological differences.

> **Note**
>
> The techniques discussed this week rely on document-feature matrices as the numeric representation of text. Next week, we will introduce another representation — embeddings — which underpin many modern text analysis methods and are fundamental to large language models.

2.2 Lab

Text preprocessing and document-feature matrices in Python.

2.3 Readings

- Grimmer and Stewart (2013)

3 Embeddings

> **Week 3**
>
> **Lecture:** Word embeddings, sequence embeddings, and document embeddings
> **Lab:** Measuring text similarity using embeddings

3.1 Lecture

Last week, we discussed document-feature matrices as one way of representing words and documents numerically. This week, we introduce another approach — embeddings — which are used for many modern text analysis methods and are also fundamental to large language models.

3.1.1 From bag-of-words to embeddings

3.1.1.1 The bag-of-words model

The bag-of-words (BoW) model is one of the simplest ways to represent text numerically. A sentence such as "The cat sat on the mat" would be represented as a vector of word counts: *the*: 2, *cat*: 1, *sat*: 1, *on*: 1, *mat*: 1. This approach has several important limitations: it loses word order ("the cat sat" and "sat the cat" would be identical), it cannot capture semantic relationships between words, and it creates very sparse, high-dimensional vectors since most words in a vocabulary will not appear in any given document.

3.1.1.2 Distributional semantics

Embeddings are grounded in the principle of distributional semantics, often summarised by the phrase: "you shall know a word by the company it keeps." The idea is that meaning is learned and defined by context — we learn to associate a particular word with a particular object or referent by encountering it repeatedly in different contexts.

3.1.2 What are embeddings?

Embeddings represent a fundamental shift from counting words to understanding their meaning through context. Unlike bag-of-words representations, embeddings are:

- **Dense vectors** (most values are non-zero)
- **Semantic** (similar words have similar vectors)
- **Compact** (typically 100–300 dimensions, compared to vocabulary-sized vectors)

Each word is mapped to a fixed-length vector. Words that appear in similar contexts have similar vector representations, and the distance between vectors corresponds to semantic distance (measured by Euclidean distance, cosine similarity, Manhattan distance, or other metrics).

3.1.2.1 Understanding vector dimensions

A 2-dimensional embedding is like coordinates on an X/Y axis. In 3D, we add a Z axis. Beyond three dimensions, it becomes difficult to visualise the space, but higher-dimensional representations are mathematically useful and capture richer semantic relationships. There are methods (such as PCA or t-SNE) that allow us to reduce this space back down to 2D or 3D for visualisation and interpretation.

3.1.3 Geometric properties of embeddings

Visualising embeddings reveals patterns in how words relate to each other. Semantically similar words cluster together: "cat" and "kitten" have high similarity (near synonyms), "cat" and "dog" have moderate similarity (same category), while "cat" and "house" have low similarity (unrelated concepts). This mirrors human intuition about word relationships.

3.1.3.1 Vector arithmetic

One striking property of embeddings is that vector arithmetic can be used to illustrate semantic relationships. The famous example is:

$$\text{king} - \text{man} + \text{woman} \approx \text{queen}$$

This shows that embeddings capture gender relationships as a consistent directional offset. Other examples include: Paris − France + Italy Rome, and terrible − bad + good great.

These patterns hold even across languages: when words from different languages are embedded in the same space, semantically similar words cluster together regardless of language.

3.1.4 Training word embeddings

Word embeddings can be trained using prediction-based or count-based methods.

Prediction-based models include:

- **Word2Vec** (Google, 2013), which has two architectures: *Skip-gram*, which predicts context words from a target word (e.g. given "cat", predict "The ___ sat"), and *CBOW* (Continuous Bag of Words), which predicts a target word from its context (e.g. given "The ___ sat", predict "cat"). Skip-gram works better for rare words, while CBOW is faster and better for frequent words. The training objective is to maximise the probability of actual context words while minimising the probability of unrelated words.
- **GloVe**, which combines the strengths of count-based and prediction-based methods.
- **FastText**, which handles out-of-vocabulary words by using subword information.

Count-based models create large matrices counting how often words co-occur, then use singular value decomposition (SVD) to reduce dimensionality while preserving important relationships. These are computationally expensive but theoretically well-understood.

In both cases, the embedding vectors are gradually adjusted during training (via backpropagation, similar to neural network training) to minimise prediction errors.

3.1.5 Bias in word embeddings

Word embeddings do not create bias, but they inherit it from the training data. For example, early embeddings trained on news corpora placed "doctor" closer to "man" and "nurse" closer to "woman" in the embedding space.

Debiasing methods have been proposed, but these can result in other kinds of distortion and will inevitably attend to some types of bias more than others. Training data in general disproportionately reflects the WEIRD demographic (Western, Educated, Industrialised, Rich, Democratic).

As social scientists, we generally do not apply debiasing methods because those differences in how social groups are discussed are often interesting to us and are frequently the object of study. However, we must be conscious of these biases and how they impact our results. We will discuss bias more extensively in Week 8.

3.1.6 Applications of embeddings

Embeddings serve two main purposes in research:

1. **As feature representations for machine learning tasks** such as text classification, sentiment analysis, and machine translation.

2. **As objects of direct study**, enabling researchers to analyse how language and meaning vary across groups, contexts, and time.

3.1.6.1 Case study: Gender and party differences in the US House

Rodriguez, Spirling, and Stewart (2023) used word embeddings trained separately on speeches by different groups of US legislators to examine how the same words are used with different meanings or connotations. While male and female legislators, and Republicans and Democrats, tend to use common words like "also" and "but" in similar ways, they diverge when it comes to more politically charged terms. In particular, male legislators use "marriage" in very different ways to female legislators, while Republicans refer to "immigration" in very different contexts to Democrats. Looking at nearest-neighbour terms in the embedding space, Democrats tend to use terms related to "reform" when discussing immigration, while Republicans tend to focus on "enforcement."

3.1.6.2 Case study: Gender in school textbooks

Lucy et al. (2020) examined how different populations are discussed in K–12 textbooks in Texas using word embeddings. They found that words related to power and achievement are more associated with terms referring to men, while words related to home life and labour are more associated with women (the labour association likely reflecting historical discussion of women entering the workplace).

3.1.7 Sentence, paragraph, and document embeddings

While word embeddings represent individual words, we can also compute embeddings for longer text sequences — sentences, paragraphs, or entire documents. These are used for clustering, topic modelling, classification, legislative text comparison, policy area classification, speaker comparison, and ideological scaling.

3.1.7.1 Case study: Party positions in UK Parliament

Rheault and Cochrane (2020) calculated party embeddings from parliamentary debates and applied PCA. The first principal component (x-axis) captured the left–right divide, while the second (y-axis) captured the government–opposition divide. The resulting plot shows how parties shifted over time, with the Thatcher years being furthest to the right and Labour moving rightward during the Blair years under New Labour. The authors also calculated embeddings for individual legislators, allowing comparisons between individual MPs rather than just between parties.

3.1.8 Embeddings beyond text

Embedding techniques are not limited to text. Audio embeddings can be used for speaker verification and song recommendations. CLIP embeddings allow text and images to be embedded in the same vector space, so that images and their descriptions appear close together in the space. Multimodal models build on this principle to process and generate across different data types.

3.2 Lab

Working with word embeddings in Python.

3.3 Readings

- Rheault and Cochrane (2020)
- Rodriguez and Spirling (2022)

4 Transformers

> Week 4
>
> **Lecture:** Basics of transformer models
> **Lab:** Fine tuning transformers for text classification

4.1 Lecture

Last week, we discussed embeddings — the fundamental numeric representations that underlie what we turn to this week: transformers. Transformers are the neural network architecture used in all major modern language models and chatbots.

4.1.1 The transformer revolution

The transformer architecture was introduced in the landmark paper "Attention Is All You Need" (Vaswani et al. 2017). It represented a revolutionary departure from previous sequential architectures such as recurrent neural networks (RNNs). Where an RNN processes a sentence like "The cat sat" one word at a time — first "the", then "cat", then "sat" — a transformer processes the entire sequence simultaneously. This parallel processing makes transformers much faster to train and run, enables them to handle much longer texts, and makes the architecture far more scalable. The original transformer is made up of two main components: an encoder (on the left of the architecture diagram) and a decoder (on the right). Together, these characteristics lead to much better performance on tasks we care about, such as text classification.

For political science, transformers offer improved understanding of complex political texts, cross-lingual capabilities for comparative analysis, and the ability to handle long documents such as legislation and treaties.

4.1.2 Attention

The key innovation of the transformer is the attention mechanism. Each word in a sequence calculates attention scores with all other words. These scores determine how much information to gather from each other word. Multiple attention "heads" can learn different types of relationships simultaneously — some might focus on grammar, others on topic relationships or semantic meaning. Crucially, this happens for all words at once, and each word can directly access information from any other word regardless of distance in the sentence.

Consider the sentence: "The bill was passed despite fierce opposition." Self-attention helps the model connect "bill" with "passed" (subject–verb), "fierce" with "opposition" (modifier–noun), and "passed" with "opposition" (action–reaction).

4.1.2.1 Self-attention

Self-attention captures relationships within a single sequence. For example, in the sentence "The animal didn't cross the street because it was too tired", the model must determine what the pronoun "it" refers to. Slight modifications to the sentence — changing "tired" to "wide", for instance — alter the attention scores for "it", shifting its reference from "animal" to "street". The intensity of the attention scores reflects how strongly the model connects each pair of words.

4.1.2.2 Cross-attention

Cross-attention captures relationships between two sequences, and is particularly important for tasks like translation. When translating "Economic growth has slowed" into German or French, the model must not only translate individual words but also attend to differences in word order and grammatical structure between languages. The attention weights (visualised as lines of varying thickness between source and target words) indicate how much attention each word in the output pays to each word in the input.

4.1.3 Embeddings in transformers

Building on what we know about embeddings from the previous week, transformers incorporate three separate types of embeddings:

- **Token embeddings** convert discrete words into continuous vectors.

- **Position embeddings** encode the location of each word in the sequence, providing information about word order. This is a critical component that gives transformers much better language understanding than previous approaches. Consider the difference between "Party A opposes Party B" and "Party B opposes Party A", or between "Increased spending and reduced taxes" and "Reduced spending and increased taxes" — position embeddings allow the model to distinguish between these.
- **Segment embeddings** help the model handle multiple-sentence tasks by marking sentence boundaries.

All three types of embedding are learned during training.

4.1.4 Transformers and LLMs

The terms "transformer" and "LLM" are often used interchangeably, but their relationship is not one-to-one. Modern LLMs generally use the transformer architecture, but transformers extend beyond language to computer vision and audio processing. Alternative LLM architectures also exist (including RNN- and CNN-based models), though these remain largely experimental and focus primarily on computational efficiency. At present, transformer-based LLMs dominate and the capabilities gap remains significant.

4.1.5 Model architectures

There are three main categories of transformer-based model, each suited to different tasks:

Encoder-only models (e.g. BERT, RoBERTa) use bidirectional context — they can attend to words both before and after a given position. They are best suited for classification, text understanding, and analysis tasks.

Decoder-only models (e.g. GPT, LLaMA) process text from left to right and are best suited for text generation, completion, and chat.

Encoder-decoder models (e.g. T5, BART) incorporate both components and are best suited for translation, summarisation, and question answering.

The way these models are trained gives them different competencies. Encoder models like BERT are trained by randomly masking words in the input and learning to fill in the blanks, which builds a deep understanding of language structure. Decoder models like GPT are trained on incomplete texts and learn to generate one word at a time, which develops strong generative capabilities.

4.1.6 Pre-training and fine-tuning

Models are first pre-trained on large-scale general datasets to create foundation models. Pre-training involves learning broad language patterns from massive corpora through objectives like masked language modelling. These foundation models can then be worked with directly in some cases, but in most cases will require some level of fine-tuning.

Fine-tuning adapts a pre-trained model for a specific task or domain using a smaller, task-specific dataset. "Domain" here refers to a specific type of text — for example, a model may be pre-trained on all the text on the internet and then fine-tuned on social media posts. Fine-tuning is much faster and requires far less data than training from scratch, because the model has already learned general language representations. A foundation model can be fine-tuned for different purposes: for instance, a base BERT model might be fine-tuned for classification or to generate general-purpose embeddings.

4.1.7 Zero-shot classification

Zero-shot classification is when a model performs a task without any fine-tuning, relying instead on the understanding of language that it acquired during pre-training. Given a piece of text and a set of candidate labels that the model has never been explicitly trained on, it determines how the input relates to each label and outputs probability scores. Encoder-decoder models can be used for zero-shot classification, and generative (decoder-only) models can also serve this purpose — we will return to generative models in Weeks 6 and 7.

4.1.8 Applications in political science

Transformers enable a range of applications in political science, including automated classification of political texts (categorising legislation by policy area, identifying key issues in parliamentary speeches), cross-lingual comparative analysis (comparing policy positions across party manifestos, analysing framing of issues in international media), and extracting structured data from unstructured text (identifying political actors and their relationships, tracking mentions of bills and policies in debates).

4.1.8.1 Case study: Calls to action and protest attendance

Rogers, Kovaleva, and Rumshisky (2019) used a BERT model fine-tuned on Russian (RuBERT) to identify calls to action in tweets. They found that increases in calls to action showed a moderate correlation with increases in protest attendance, suggesting that transformer-based text classification may be useful as a predictor of protest movements.

4.1.8.2 Case study: Framing of migration on Twitter

Mendelsohn, Budak, and Jurgens (2021) used a RoBERTa model (an improvement on the original BERT) to classify immigration-related tweets according to how they are "framed." They found that tweets from US users about migration tended to focus on law-and-order implications and economic threat, while tweets from EU users tended to frame migrants as victims of war and globalisation, but also discussed migration in the context of cultural identity and threats to national cohesion.

4.1.9 Considerations

There are several important considerations when working with transformer models. Computational requirements are significant: these models demand high processing power and substantial memory. Training data biases carry over into the model, and domain adaptation remains a challenge. Interpretability is also a concern — the complex attention patterns within these models give them something of a "black box" nature. We will discuss parameter-efficient fine-tuning (PEFT) and adapters in Week 9, and interpretability in more depth in Week 10.

4.2 Lab

Content to be added.

4.3 Readings

- Widmann and Wich (2023)
- Laurer et al. (2024)

5 Validation and Performance Measurement

> **Week 5**
>
> **Lecture:** Performance metrics, human coding, and cross-validation
> **Lab:** Measuring model performance with `scikit-learn`

5.1 Lecture

Building on last week's deep dive into transformer architectures, we now turn to the question of how we know whether a model is actually performing well. This week focuses on robust methods for validating LLM outputs, quantifying performance, and ensuring reproducibility. These are not abstract concerns — they are essential for credible, impactful application of LLMs to political and social science research questions.

The increasing ease-of-use of LLMs makes validation more important, not less. As we discussed last week, zero-shot classification allows a researcher to hand a model a set of candidate labels and a corpus of text and receive classifications without any training data at all. The barrier to entry has effectively collapsed: what once required substantial labelled datasets, machine learning expertise, and weeks of model development can now be done in an afternoon with a few lines of code. But this convenience is deceptive. It is easy to throw a set of candidate labels at a model and get results without having built out the concepts behind those labels — without reducing their overlap, parsing out edge cases, or ensuring that they translate well to the specific domain under study.

This creates a tension with foundational principles of measurement in political science. In the standard conception of measurement validity (Adcock and Collier 2001), researchers must transform broad "background" concepts into precise, systematised constructs through careful operationalisation. When we bypass this process and rely on an LLM's pre-trained understanding of a label, we risk measuring something other than what we intend. Halterman and Keith (2025) illustrate this problem with a concrete example: different research projects operationalise "protest" in substantively different ways. The Crowd Counting Consortium's definition of protest excludes labour strikes, while the CAMEO event ontology's definition includes them. If a researcher simply passes the label "protest" to an LLM without specifying which definition they mean, the model will rely on whatever internal representation of "protest" it has learned from its pre-training data — and if those training contexts included strikes, the

resulting classifications may include labour strikes as protests, leading to incorrect substantive conclusions. The label looks right, the output looks plausible, but the measurement is invalid.

The same problem arises across many political science concepts. "Populism", "polarisation", "democratic backsliding" — these are all terms with contested or context-dependent meanings that an LLM may interpret differently from how a researcher intends. Validation is the process by which we check whether a model's outputs actually correspond to the constructs we care about, and it is the only safeguard against plausible-looking but misleading results.

5.1.1 Validation in LLM research

Validation for LLMs is uniquely challenging compared to traditional quantitative political science methods. Surveys and experiments have well-established validation playbooks: we can assess question wording, sample representativeness, treatment delivery, and so on. LLMs, by contrast, are often opaque systems with inconsistent outputs across runs due to inherent randomness, making it harder to diagnose issues. Hallucination (outputting fabricated information) and bias (reflecting societal stereotypes) are known failure modes that can be difficult to quantify. Robust validation methodologies are therefore essential if we are to have confidence in LLM-based findings.

Three key validation concepts underpin this effort: establishing ground truth, computing performance metrics, and conducting human evaluation.

5.1.2 Ground truth

The ideal validation setup requires a clear "ground truth" against which to evaluate model outputs. Ground truth can be approximated through expert annotations, historical data, or rigorously verified facts, but all of these have limitations.

Political science phenomena are often complex, subjective, and open to interpretation, making ground truth particularly elusive in our field. Annotator background, textual source, and time period can all shape what counts as "true." The meaning of concepts can shift over time, and context-dependent language poses further challenges. It is important to engage deeply with primary sources and existing literature to contextualise model performance, and to always consider the potential for bias in both inputs and outputs.

5.1.3 Confusion matrices

The confusion matrix is the fundamental tool for understanding classifier performance. It is a table that cross-tabulates actual classes (ground truth) in rows against predicted classes (model outputs) in columns. Diagonal entries represent correct predictions — true positives and true

negatives — while off-diagonal entries represent errors: false positives (predicted positive but actually negative) and false negatives (predicted negative but actually positive).

The confusion matrix provides a structured way to identify not just how much a model gets right, but the specific types of errors it makes. Which errors matter most depends on the application. In spam detection, a false positive (a legitimate email filtered as spam) is typically worse than a false negative (a spam email reaching the inbox). In medical screening, the opposite is true: a false negative (missing a real case) is far more costly than a false positive (flagging a healthy patient for follow-up).

5.1.4 Performance metrics

Accuracy, precision, recall, and F1 score are all computed from the raw counts in the confusion matrix.

Accuracy is the proportion of predictions that are correct: $(TP+TN)/(TP+TN+FP+FN)$. It is intuitive but can be misleading with imbalanced classes. If 90% of emails are non-spam, a model that always predicts "non-spam" achieves 90% accuracy while being entirely useless.

Precision answers the question: when the model predicts positive, how often is it correct? It is calculated as $TP/(TP + FP)$ and measures the quality of positive predictions.

Recall answers the question: out of all the actual positive instances, how many did the model successfully identify? It is calculated as $TP/(TP + FN)$ and measures the completeness of positive predictions.

F1 score is the harmonic mean of precision and recall: $2 \times (Precision \times Recall)/(Precision + Recall)$. It provides a single balanced metric that is useful for comparing models, particularly when there is no strong reason to prioritise precision over recall or vice versa.

Note that "positive" and "negative" are defined by the problem — it is important to think carefully about which class is which when interpreting these metrics.

5.1.4.1 The precision–recall tradeoff

There is often a tradeoff between precision and recall; it is rare to maximise both simultaneously. A model with high precision but low recall is like a small, tightly woven net: it catches mostly true cases but misses many. A model with high recall but low precision casts a wide net, identifying most true cases but also picking up many false positives.

Which to prioritise depends on the domain and the relative costs of different error types. Spam detection typically favours high precision (it is acceptable to miss some spam, but very bad to filter out legitimate emails). Medical screening favours high recall (catching as many real cases as possible is the priority, since follow-up tests can weed out false positives). Most problems

fall somewhere in between, and researchers need to think carefully about these error tradeoffs in the context of their specific research question.

5.1.4.2 Accuracy vs. precision: a visual analogy

The distinction between accuracy and precision can be illustrated with a dartboard analogy. The bullseye represents the ground truth. Accuracy measures overall closeness of the darts (predictions) to the bullseye, while precision captures how tightly grouped the darts are. A model can have high accuracy while being imprecise (predictions scattered around the bullseye), or high precision while being inaccurate (tightly grouped predictions that are systematically off-centre). A precise but inaccurate model may have a systematic bias that could potentially be corrected through recalibration. This connects to the bias–variance tradeoff: balancing model complexity to avoid both underfitting (high bias) and overfitting (high variance) for better generalisation.

5.1.5 Multiclass averaging methods

Many political science applications involve multiclass or multilabel classification rather than simple binary problems. When computing metrics such as precision, recall, and F1 across multiple classes, there are several ways to aggregate them.

Macro averaging computes each class's metric independently and takes the unweighted mean. This gives equal emphasis to all classes regardless of frequency, making it a good choice when all classes are equally important and performance on minority classes matters.

Micro averaging pools all the true positive, false positive, and false negative counts across classes and computes metrics on the pooled counts. This implicitly weights by class frequency and naturally emphasises the most common classes. It is useful when the overall volume of correct predictions is more important than class-level performance.

Weighted averaging computes per-class metrics and then takes a weighted average, with weights proportional to the number of instances in each class. This accounts for class imbalance while allowing each class to contribute to the final metric.

To illustrate: consider a three-class problem where Class A has 100 instances (F1 = 0.90), Class B has 50 instances (F1 = 0.80), and Class C has 10 instances (F1 = 0.60). The macro F1 would be $(0.90 + 0.80 + 0.60)/3 = 0.77$, while the weighted F1 would be $(100 \times 0.90 + 50 \times 0.80 + 10 \times 0.60)/160 = 0.84$. Notice how macro averaging treats Class C equally despite its small size, pulling the average down, while weighted averaging gives more influence to the larger classes.

In many political science settings, weighted averaging is a reasonable default because it captures class prevalence without completely ignoring rare classes. But the choice should always

be guided by the research question: use macro if all classes are equally important, micro if overall performance is key, and weighted for a tailored balance.

5.1.6 Standard benchmarks vs. domain validation

Generic language benchmarks such as GLUE, SuperGLUE, SQuAD, and CNN/Daily Mail are valuable for comparing LLMs on broad tasks, but they have limitations for political science applications. Our work often involves domain-specific language, concepts, and historical knowledge that general-purpose models and datasets do not capture well. Even moving from parliamentary speech to legislation can pose validity problems for a model trained on one type of text.

Bespoke validation datasets and performance measures tailored to political science constructs and research questions are often necessary to build real trust in a model. Developing these takes effort, but it is essential for moving beyond "off the shelf" use of LLMs. This means we need domain-specific labelled (hand-coded) data to validate against.

5.1.7 Task-specific evaluation metrics

Standard classification metrics such as accuracy, precision, recall, and F1 are a starting point, but researchers should also consider task-specific measures. BLEU evaluates machine translation quality based on n-gram overlap between model outputs and reference human translations. ROUGE assesses summarisation quality based on overlapping word sequences between model and reference summaries. Perplexity measures a language model's uncertainty in predicting the next word.

These metrics give a more focused sense of how well a model is performing the specific task at hand, beyond generic predictive power. Human evaluation — having raters assess output fluency, coherence, and relevance — is also valuable despite being labour-intensive. Careful qualitative error analysis can surface insightful patterns about model failure modes, helping to build a more nuanced picture than summary metrics alone and pointing to potential fixes.

5.1.8 Crowdcoding

Crowdsourcing (or "crowdcoding") can be a powerful tool for efficiently building domain-specific validation datasets. Services such as Amazon Mechanical Turk allow researchers to hire annotators at scale, which is especially valuable for labelling tasks that benefit from many judgements — for example, labelling sentences as claims, identifying political parties, or rating sentiment.

Crowdsourcing leverages the diversity of the crowd to reduce individual annotator biases. However, quality control is critical, especially for nuanced political science concepts. Researchers

cannot simply assume high-quality labels from the outset. Careful training of annotators, very precise annotation guidelines with worked examples, and ongoing monitoring of outputs are all essential.

5.1.9 Codebooks and annotation

Codebooks are key for making annotation efforts reproducible and consistent across multiple annotators. They operationalise abstract constructs into discrete labels that can be applied systematically, and they use examples and decision rules to ensure that everyone interprets edge cases in the same way.

A good codebook has clear definitions of all categories and anticipates likely points of confusion or disagreement, providing guidance on how to resolve them. Developing a codebook is often an iterative process: draft the codebook, try coding with it, identify points of divergence, discuss and amend, and repeat. This can be time-consuming but is critical for producing high-quality labelled data that others can make sense of and reuse.

5.1.10 Intercoder reliability

Inter-annotator agreement metrics quantify the degree of consistency between multiple human raters — a form of reliability measurement.

Percentage agreement is the simplest measure: the proportion of items on which coders agree. It is intuitive but does not account for chance agreement. With only two categories, even random guessing would produce agreement roughly 50% of the time.

Cohen's kappa corrects for chance agreement and is designed for two raters working with categorical data. It is widely used but limited to pairwise comparisons.

Krippendorff's alpha is more flexible: it handles any number of raters, works with different data scales (nominal, ordinal, interval), and accounts for missing data.

As a rough guide for interpretation: values below 0.4 indicate poor agreement, 0.4–0.6 moderate agreement, 0.6–0.8 substantial agreement, and above 0.8 near-perfect agreement. High agreement suggests that annotations are reproducible and trustworthy; low agreement points to ambiguity in categories or gaps in annotator training. However, agreement alone is not sufficient — coders could agree consistently on a flawed set of labels. Metrics should always be combined with human inspection.

When choosing a reliability metric, percentage agreement is acceptable for quick checks and clearly defined, non-overlapping categories. Cohen's kappa is appropriate for two independent raters doing categorical labelling. Krippendorff's alpha is the most general option and is preferable when there are multiple raters, missing values, or ordinal/interval data. When in doubt, it is better to use a chance-corrected metric.

5.1.11 Human evaluation challenges

Human evaluation of LLM outputs is invaluable for assessing quality but is susceptible to several cognitive biases. Anchoring effects mean that human ratings can be swayed by the order in which outputs are seen — if raters start with strong outputs, mediocre ones later may seem worse than they are. Priming effects arise when raters are shown the "correct" output before rating, biasing their perception. Fatigue effects cause rating quality to drift over time as raters tire or lose concentration, introducing noise.

Careful experimental design can mitigate these problems: randomising presentation order, separating correct answers from submissions to be rated, and enforcing breaks. But some subjectivity is inevitable, even with the best designs. Transparency is key: report inter-annotator metrics, distribute annotation guidelines, and acknowledge limitations. Human evaluation scores should be one piece of the overall assessment alongside more quantitative metrics.

5.1.12 Open science and reproducibility

Open science practices are essential for reproducibility — making it possible for others to validate and extend findings. Sharing data and code should be the norm: not just final processed data, but raw inputs, processing scripts, model training code, and everything in between. Detailed methods sections that outline key steps and decision points, supplemented by digital appendices recording hyperparameter settings, random seeds, and other fine-grained details, enable scrutiny and follow-on work by the broader community.

A wide range of sharing platforms are available, including GitHub, the Open Science Framework (OSF), and Dataverse. Many venues now require data and code sharing, but it is worth doing even when not strictly mandatory. Researchers should also consider intellectual property, data privacy, and ethical issues around sharing — sensitive information should be scrubbed and necessary permissions obtained.

5.1.12.1 Preregistration

Preregistering studies makes it clear which analytic choices were made ahead of time versus after seeing the data. A preregistration typically includes clearly specified research questions, data collection and annotation procedures, exclusion criteria, and planned analyses.

Preregistration does not prevent exploratory analysis, but it makes the line between confirmatory and exploratory work explicit. This helps avoid HARKing (hypothesising after results are known), p-hacking, and overfitting to noise. It is not just a box to check — it is a valuable tool for making studies more credible and inviting replication.

Preregistration can be challenging for LLM projects that are intrinsically iterative or exploratory. In such cases, researchers might preregister key decision points or write up results

as a registered report. Many preregistration templates are available on platforms like OSF and AsPredicted.

5.1.12.2 Version control with Git

Version control is essential for managing complex codebases and collaborating effectively. Git, together with web-based platforms like GitHub, is the standard tool. It tracks changes, supports branching and merging, and integrates well with other reproducibility tools.

The basic Git workflow involves four areas: the working directory (where files are edited), the staging area (where changes are prepared for commit), the local repository (the commit history on the local machine), and the remote repository (the shared project history hosted online). The core commands map to movements between these areas: `git add` stages changes, `git commit` creates a snapshot, `git push` uploads commits to the remote, and `git pull` downloads and integrates remote changes.

Git models project history as a tree. Each commit is a node, and branches split off and merge back in. Regular commits with clear messages create a log of project evolution. Tagging particular commits can mark important milestones such as paper submissions or data freezes. Diffs show line-by-line changes between versions, making it straightforward to track what changed, when, and why.

Branching allows parallel development and experimentation without disrupting the main codebase. Each branch is a separate workspace for developing a feature, fixing a bug, or exploring an idea. Branches can be merged back into the main branch when ready, with Git intelligently combining changes and flagging conflicts for manual resolution. This is also useful for code review: a collaborator can share a branch, receive feedback, make changes, and merge once the work is approved.

5.1.12.3 GitHub

GitHub is the dominant platform for collaborative coding and project management. It provides hosted Git repositories with free public and paid private options, and its web interface adds many convenient features: an issue tracker for reporting bugs and discussing tasks, project boards for organising work, wikis for shared documentation, and GitHub Pages for hosting static websites. GitHub also has a large open-source community, making it a natural first port of call when looking for code or sharing your own.

5.1.12.4 Reference management and document preparation

Reference managers such as Zotero, Mendeley, and BibTeX minimise citation formatting hassles, store PDFs, and synchronise bibliographies across devices. Plain-text formats such as LaTeX, Markdown, and Jupyter notebooks separate content from presentation, making writing modular and helping to avoid over-fixation on formatting during drafting. These tools, combined with version control and automated backups, form the infrastructure of a reproducible research workflow.

5.2 Lab

Content to be added.

5.3 Readings

- Birkenmaier, Lechner, and Wagner (2024)

6 Generative Language Models I

Week 6

Lecture: Text analysis research pipeline, generative model fundamentals, and applications in political science
Lab: Using generative language models for text analysis

6.1 Lecture

This week marks a transition from classification-focused approaches to generative language models. Before discussing these models in detail, it is important to situate them within the broader text analysis research pipeline.

6.1.1 The text analysis research pipeline

Whether using traditional machine learning, transformer-based classifiers, or generative models, text analysis research in political science typically follows the same general pipeline. Understanding this pipeline is essential for designing rigorous studies and for your course projects.

Theory and hypothesis generation. Following open science practices, theory building and hypothesis generation should come before data collection and exploration. This is when you would typically preregister your research, including questions, hypotheses, and design. It is critical to maintain a theory-driven rather than data-driven approach, especially with powerful generative tools available that might tempt researchers into exploratory fishing expeditions.

Data collection and preprocessing. Text data comes in many forms: speeches, social media content, news articles, policy documents, and more. Preprocessing steps include tokenisation (breaking text into words or subwords), cleaning (removing special characters and formatting), and normalisation (lowercasing, lemmatisation). All preprocessing steps should be documented for reproducibility.

Manual annotation. This step should be conducted before any classification with a model to ensure clear concept definition. It is particularly important when working with generative models to ensure that definitions are driven by the researcher's theoretical understanding, not

the model's. Through annotation, researchers encounter edge cases that provide insight into where a model might succeed or fail. Annotation at this stage helps maintain conceptual clarity throughout the research process.

Model training and classification. Modern approaches include fine-tuning transformer models on annotated data. Validation on unseen data is critical — never evaluate on your training data. Cross-validation helps ensure robustness of performance evaluation.

Aggregation and quantification. After classification, results often need to be aggregated to meaningful analytical units. If classifying sentiment at the sentence level, for instance, results may need to be aggregated to the document or speaker level. Aggregation methods include simple counts or proportions, weighted approaches based on confidence scores, and time-series representations for temporal analysis. This step connects model outputs to testable hypotheses.

Hypothesis testing and interpretation. The final step applies standard statistical techniques to aggregated model outputs. Common approaches include regression models, multilevel models, and time-series analysis. It is critically important to connect statistical findings back to the original theory. Models are tools to help answer questions, not ends in themselves.

Notice the cyclical nature of this pipeline: validation feeds back into model training, and findings may prompt refinement of earlier steps.

Worked example

Consider a study of emotions in party manifestos. Theory from populist communication research suggests that populist parties leverage negative emotions. The research question might be: do populist parties utilise fear appeals more than mainstream parties? The hypothesis: populist manifestos will contain higher proportions of fear language. Data collection involves gathering party manifestos from multiple countries and elections, from both populist (e.g. AfD, Lega) and mainstream parties. Manual annotation requires developing a codebook defining "fear language" with examples of threats, dangers, and risks, then annotating a subset of sentences. Model training involves building a classifier to identify fear language and validating on held-out data. Aggregation calculates the proportion of fear content by party type, country, and election year. Hypothesis testing then compares fear language prevalence statistically, controlling for economic conditions and ideology, and interpreting results in relation to populist communication theory.

6.1.2 Generative language models

Generative models fundamentally differ from classification models in their ability to produce novel text. This capability creates both opportunities and challenges for research applications.

6.1.2.1 Decoder-only architectures

Most modern generative LLMs — including ChatGPT, Claude, Llama, and Mistral — use a decoder-only architecture. This is a key distinction from the encoder-decoder models discussed in Chapter 4. A decoder-only model processes text sequentially, predicting one token at a time based on all previous tokens, using the self-attention mechanism to consider the full preceding context. This architecture is particularly efficient for text generation, with a simpler training process and inference pipeline compared to encoder-decoder models.

6.1.2.2 Training and compute

Pre-training happens on massive text corpora — hundreds of terabytes drawn from the web, books, scientific papers, and other sources, amounting to trillions of tokens of text. The computational resources involved are staggering: GPT-4 is estimated to have cost anywhere from $30 million to over $100 million to train, using thousands of specialised GPUs over several months.

After pre-training, foundation models typically undergo instruction tuning with reinforcement learning from human feedback (RLHF). This process aligns models with human preferences and reduces harmful outputs. RLHF works by first fine-tuning the model on a dataset of instructions paired with desired responses, then collecting human feedback by having people rank different model outputs, training a separate reward model that learns to predict human preferences, and finally using reinforcement learning to optimise the model against this reward model.

The environmental impact of training these models is significant, with electricity consumption equivalent to that of small towns. These resource requirements create accessibility barriers for academic researchers.

6.1.2.3 Model size

Model size is measured in parameters — the weights and biases that can be adjusted during training. Sizes have grown exponentially, from BERT-large's 340 million parameters in 2018 to GPT-3's 175 billion in 2020 and models estimated at over a trillion parameters in more recent years. Models can be roughly categorised as small (1–7 billion parameters, e.g. Mistral 7B), medium (13–70 billion, e.g. Llama 3 70B), or large (100 billion and above, e.g. Llama 3 405B, DeepSeek R1 at 685 billion, or GPT-4 estimated at around 1.8 trillion).

Generally, larger models perform better across tasks, but with diminishing returns. Smaller models are now competitive for many tasks thanks to improved training techniques. The size–performance tradeoff is particularly relevant for researchers with limited compute, as resource constraints often determine which models can realistically be used.

6.1.2.4 Open source vs. closed source

Closed-source models such as GPT-4, Claude, and Gemini are proprietary: access is provided through APIs, and model weights are treated as trade secrets. They have historically had a performance edge but lack transparency. Open-source models such as Llama 3, Mistral, Gemma, and DeepSeek are publicly available, can be run locally, and can be modified and fine-tuned. Recent open-source releases have substantially closed the performance gap.

For research, open-source models offer important advantages: the ability to run locally without API costs, transparency about training data and methods, the possibility for modification and fine-tuning, and crucially, reproducibility of results. However, leading closed models still maintain advantages in certain capabilities.

6.1.2.5 APIs

Application Programming Interfaces (APIs) provide programmatic access to models without the need to run them locally. Major providers include OpenAI (GPT-3.5, GPT-4), Anthropic (Claude), and Google (Gemini). For researchers, several considerations apply: costs accumulate quickly with large datasets (charged per token for both input and output), rate limits restrict throughput, data privacy and storage policies vary, and models may change without notice, creating a "moving target" problem that complicates reproducibility. Budget planning is essential for research projects using commercial APIs.

6.1.3 Generative models in political science

6.1.3.1 Tailored political persuasion

Recent research has demonstrated that LLMs can generate very persuasive political messages (Hackenburg and Margetts 2024), and that these messages can be individually tailored using personal data to increase their persuasive effect (Timm, Talele, and Haimes 2025). Applications include campaign communications, policy explanations, and voter outreach. However, the ethical considerations are significant: there are risks of manipulation through personalised persuasion, privacy implications related to data collection, and potential impacts on democratic transparency. Persuasion techniques tend to seem helpful when used by one's own side but manipulative when used by opponents.

6.1.3.2 Simulating opinion

The "silicon samples" approach uses LLMs with fictional personas and backstories to simulate survey responses (Argyle et al. 2023). Researchers have also used LLM agents to study how misinformation evolves and spreads on social media (Liu et al. 2024). These approaches

offer rapid hypothesis testing and cost-effectiveness compared to traditional surveys, but face limitations including sociodemographic and political biases in model outputs and the difficulty of validating against ground truth. This approach should complement rather than replace human subject research.

6.1.3.3 Simulating behaviour

LLMs can simulate political actors in various contexts: coalition negotiations in parliamentary democracies (Moghimifar et al. 2024), legislative behaviour in the US Senate (Baker and Azher 2024), and even hypothetical interactions between human and alien civilisations (Xue et al. 2025). These simulations are particularly valuable for exploring counterfactual scenarios that are difficult to study empirically. However, models may struggle with strategic reasoning, risk reinforcing stereotypes, and are difficult to validate against actual behaviour. This approach is most appropriate for exploratory research and theory development, and should be combined with other methodological approaches.

6.1.3.4 Creating training data

Generative models can create synthetic training data for downstream models. This is particularly useful for augmenting small datasets where human annotation is expensive, creating balanced datasets for rare categories, and generating counterfactual examples. An important distinction applies here: the researcher studies real-world data using a model trained on synthetic data, rather than directly studying synthetic data. Quality control and validation are essential when using this approach.

6.1.3.5 Generative models in text analysis

Beyond synthetic data generation, generative models can assist with classification of political content, creating text embeddings, labelling identified clusters, and summarising large document collections. However, validation against human judgements remains essential, and model parameters and prompts must be documented for reproducibility. These tools should augment rather than replace careful research design and methodology.

6.2 Lab

Content to be added.

6.3 Readings

- Gilardi, Alizadeh, and Kubli (2023)
- Argyle et al. (2023)

7 Generative Language Models II

Week 7

Lecture: Prompt engineering, hyperparameters, and LLMs vs. human annotation
Lab: Prompt engineering and API usage in Python

7.1 Lecture

This week continues our exploration of generative language models, with a focus on practical prompt engineering techniques and the hyperparameters that control model output. These skills are immediately applicable to course projects and are critical for anyone using LLMs in computational social science research.

7.1.1 LLMs vs. human annotators

Before turning to prompt engineering, it is worth considering how LLMs compare to human annotators for text classification tasks. Gilardi, Alizadeh, and Kubli (2023) compared the performance of ChatGPT against trained annotators and Amazon Mechanical Turk crowd workers across several datasets, measuring both accuracy and intercoder agreement. Their results showed that LLMs often match or exceed human performance on straightforward tasks such as relevance classification, particularly when temperature settings are kept low (e.g. 0.2 rather than 1.0). However, human validation remains essential for more complex judgements involving framing or nuanced conceptual distinctions. These findings have important implications for research scalability and efficiency, but they do not eliminate the need for careful validation.

7.1.2 Prompt engineering

Prompt engineering is the practice of optimising inputs to get better outputs from generative models. It is both an art and a science, requiring experimentation and creativity alongside systematic evaluation. Small changes to a prompt can dramatically affect output quality — the same sentiment analysis task can yield verbose and inconsistent results with one prompt

formulation, and clean, structured outputs with another. The goal is reliable, consistent outputs that meet research standards.

7.1.2.1 Prompting fundamentals

Three simple techniques often outperform switching to a larger model:

Specificity. Vague prompts lead to vague answers. Instead of "Analyse this speech," a more effective prompt would be "Analyse the economic policy positions in this speech using the CAP codebook." Precision about requirements, categories, and expected output format substantially improves results.

Reducing hallucination. Explicitly instructing models to indicate uncertainty reduces hallucination. Telling the model to respond with "I don't know" or "N/A" when it is unsure helps prevent fabricated outputs.

Instruction order. LLMs exhibit primacy and recency effects: they tend to attend most carefully to instructions placed at the beginning or end of a prompt, and may lose track of information in the middle. Place the most important instructions accordingly.

7.1.2.2 Few-shot prompting

Few-shot prompting teaches the model by example. In a zero-shot prompt, no examples are provided and the model relies entirely on pre-training. In a one-shot prompt, a single example guides the response format. In a few-shot prompt, multiple examples establish a clear pattern. Models can learn patterns from just two or three examples, and few-shot prompting is particularly effective for specialised tasks, domain-specific vocabulary, and ensuring consistent formatting in research applications. Examples should represent the full range of expected inputs.

Dynamic few-shot prompting is an advanced variant. Rather than using fixed examples for all queries, it selects examples from a larger pool based on their similarity to the current query. This requires computing semantic similarity between the query and available examples and selecting the most relevant ones for each prompt. It is more complex to implement but often more effective when inputs are diverse.

7.1.2.3 Prompt components

A well-structured complex prompt may include several components, each serving a specific purpose: a **persona** that establishes expertise and role (e.g. "You are an expert in large language models"), an **instruction** that states the main task, **context** that provides background and constraints, a **format** specification for the desired output structure, an **audience** definition,

a **tone** directive, and the **data** to be processed. Not all elements are needed for every task, but research applications often require more structured prompts than casual use.

In code, a modular approach defines each component as a separate variable. The final prompt is assembled by concatenating the components. This makes it easy to experiment by changing one component at a time, improves maintainability, and supports systematic testing of component variations to identify optimal configurations. The final prompt structure should be documented in research methods sections.

7.1.2.4 Chain-of-thought prompting

Chain-of-thought (CoT) prompting asks the model to show its reasoning steps explicitly. Rather than producing an answer directly, the model walks through the logic that leads to its conclusion. This is particularly effective for mathematical problems, logical reasoning, multi-step analysis, and policy analysis with multiple interacting factors. CoT prompting improves accuracy and provides transparency into the model's reasoning process. It can be combined with few-shot examples: providing worked examples that include reasoning steps teaches the model to apply the same approach to new problems.

7.1.2.5 Self-consistency

Self-consistency is a technique for improving accuracy on difficult problems. The approach generates multiple independent solutions to the same problem (each using chain-of-thought reasoning), then takes a majority vote to determine the final answer. This reduces the impact of random errors in reasoning and is particularly effective for classification of ambiguous cases. The tradeoff is increased computation time and cost, but for high-stakes applications requiring reliability, the improvement in accuracy can justify the expense.

7.1.2.6 Iterative prompt development

Prompt engineering is inherently iterative. The process mirrors the scientific method: start with a hypothesis (a prompt that should improve output), test it (run the prompt), analyse the results, and refine. Starting simple and adding complexity as needed, while documenting iterations to understand what works and why, is the most effective approach. This process is critical for developing reproducible research methods.

7.1.2.7 Structured output

For research applications, generative LLM responses are often most useful in machine-readable formats. Common formats include JSON, XML, and YAML. Structured outputs are easier to validate, analyse, and integrate into automated processing pipelines, and they ensure consistent format across multiple queries. When prompting for structured output, always demonstrate the desired format in the prompt, include examples with the exact keys and structure expected, and clearly specify data types for each field.

7.1.2.8 System and user prompts

When interacting with models through APIs, prompts are typically divided into two components. The **system prompt** defines the persona, expertise, and overall guidelines and constraints. The **user prompt** contains the specific text to be analysed and may include examples or criteria. This separation enables consistent behaviour across multiple queries and reusable system prompts for related tasks. For research, maintaining consistent system prompts and documenting both components is important for reproducibility. Not all models support this separation, in which case everything can be included in a single prompt. Whether separating system and user prompts makes a meaningful difference to performance is not yet entirely clear.

7.1.3 Hyperparameters

Generative language models are trained to predict the next token given a sequence of previous tokens. When generating text, the model produces a probability distribution over possible next tokens and selects one according to that distribution. Several hyperparameters control how this selection is made, and they significantly affect model outputs.

7.1.3.1 Temperature

Temperature controls the randomness of token selection. Technically, it is a scaling factor applied to the raw model outputs (logits) before they are converted into probabilities. At low temperature, the probability distribution becomes sharper: the model focuses on the highest-probability tokens, producing more deterministic, predictable, and conservative responses. At high temperature, the distribution flattens: lower-probability tokens have a greater chance of being selected, producing more diverse, creative, and unexpected responses. A temperature of 0 yields completely deterministic output (greedy decoding, always selecting the most probable token), while a temperature of 1 samples directly from the unmodified probability distribution. For most research classification tasks, low temperature settings are preferable.

7.1.3.2 Top-p (nucleus sampling)

Top-p, also known as nucleus sampling, offers an alternative way to control randomness. It selects from the smallest set of tokens whose cumulative probability exceeds the threshold p. At low top-p, only the most likely tokens are considered, producing focused and conservative outputs. At high top-p, a wider range of tokens enters the candidate pool, leading to more diverse vocabulary. Top-p often provides more intuitive control than temperature and complements it. Values typically range from 0.1 to 1.0.

7.1.3.3 Choosing settings

The appropriate settings depend on the task. For brainstorming or exploratory work, high temperature and high top-p produce diverse and creative outputs. For formal writing or email generation, low temperature and low top-p yield predictable, focused results. For creative writing, high temperature with low top-p allows creativity while maintaining coherence. For translation or technical tasks, low temperature with high top-p provides accuracy with vocabulary variety. For research classification, low temperature is almost always appropriate. These settings should be documented in research methods sections, and researchers should experiment to find optimal configurations for their specific tasks.

7.1.4 Conceptual clarity

The convenience and scalability of LLMs for text classification does not ensure accuracy. Traditional validation methods — the kind discussed in Chapter 5 — remain essential when using LLMs instead of, or alongside, human coders. Clear conceptual definitions are even more critical with generative models: the researcher should never assume that the model understands a concept in the same way they do. Outputs should always be validated against human judgements, established measures, and ground truth when available, and validation procedures should be documented thoroughly.

7.2 Lab

Content to be added.

7.3 Readings

- Törnberg (2024)
- Hackenburg and Margetts (2024)

8 Bias in LLMs

> **Week 8**
>
> **Lecture:** Defining, measuring, and mitigating bias in AI systems and LLMs
> **Lab:** Probing and measuring bias in language models

8.1 Lecture

We touched briefly on bias in word embeddings in Chapter 3 and noted the importance of validation throughout the preceding weeks. This week, we focus on bias in AI systems more broadly and in LLMs specifically, covering how bias is defined, where it comes from, how it can be measured and mitigated, and the political and social controversies that surround these efforts.

8.1.1 Defining bias in AI systems

"Bias" in the context of AI carries two distinct meanings. **Statistical bias** refers to systematic deviation from a target value — a model that consistently overestimates or underestimates something. **Normative bias** refers to unfair or unjust differential treatment of individuals or groups.

These two forms of bias can be in tension with each other. Including parental income as a predictor in a criminal justice risk assessment might improve statistical accuracy (reducing statistical bias), but it would also mean holding prejudice against individuals based on their socioeconomic background (increasing normative bias).

The harms that flow from biased AI systems can be categorised as **representational harm** (reinforcing stereotypes or rendering certain groups invisible) and **allocational harm** (unfair distribution of resources or opportunities across groups). Bias in AI is fundamentally a socio-technical phenomenon: technical systems embedded in social contexts that reflect and potentially amplify existing societal biases.

8.1.2 Examples of algorithmic bias

Several high-profile cases illustrate the range of ways bias manifests in AI systems.

8.1.2.1 Recruitment

Amazon built an automated CV screening tool trained on patterns from résumés submitted to the company over a ten-year period. Because tech hiring had been historically male-dominated, the algorithm learned to penalise résumés containing words associated with women — downgrading candidates who attended women's colleges or had terms like "women's chess club" on their résumés. Amazon attempted to modify the algorithm but ultimately abandoned the project in 2018 when they could not guarantee it would not find other proxies for gender. The system did not invent gender bias; it reflected and intensified existing patterns of discrimination in the training data.

8.1.2.2 Criminal justice

The COMPAS algorithm, used to predict recidivism risk in the US justice system, was found to have significant racial bias: it was roughly twice as likely to falsely label Black defendants as high risk compared to white defendants, while white defendants were more likely to be incorrectly labelled low risk. The company behind COMPAS disputed this analysis, arguing that their system satisfied a different mathematical definition of fairness — equal positive predictive value across groups. This case highlights how different mathematical definitions of fairness can be mutually incompatible, creating fundamental tradeoffs in system design.

Similarly, predictive policing systems have been criticised for creating feedback loops. When algorithms direct police to patrol certain neighbourhoods based on historical crime data, they generate more arrests in those areas, which then reinforces the algorithm's focus on those neighbourhoods — a self-fulfilling prophecy that can disproportionately impact communities that have historically experienced higher rates of policing.

8.1.2.3 Facial recognition

Research by Joy Buolamwini and Timnit Gebru demonstrated that leading facial recognition systems had error rates of less than 1% for light-skinned men but up to 35% for dark-skinned women. These disparities arise partly from training data that overrepresents certain demographics. When such systems are deployed in contexts like law enforcement, the consequences of misidentification fall disproportionately on already marginalised groups.

8.1.2.4 LLM-generated text

A 2023 study found that when asked to generate reference letters, LLMs produced more formal, positive, and agentic language for male candidates, emphasising individual contributions, while letters for female candidates emphasised teamwork and used more tentative language. This mirrors well-documented patterns in human-written reference letters and shows how LLMs can perpetuate subtle biases even in seemingly neutral text generation tasks.

8.1.2.5 Adversarial manipulation

In 2016, Microsoft released Tay, a Twitter chatbot designed to learn from user interactions. Within 24 hours, users had taught it to produce racist, antisemitic, and misogynistic content, forcing Microsoft to take it offline. While this example predates modern LLMs, it demonstrates the vulnerability of language models to learning harmful patterns from interactions — a challenge that persists in more subtle forms with today's more sophisticated systems.

8.1.3 Sources of bias in LLMs

Bias in LLMs emerges from multiple sources throughout the development pipeline, and understanding these sources helps in thinking systematically about where and how to intervene. It is not simply a case of "biased data in, biased results out."

Training data bias. LLMs are trained on vast corpora of internet text, which reflect and sometimes amplify societal biases. This includes over- and underrepresentation of certain perspectives, historical biases in published content, and the predominance of certain languages and cultural contexts.

Annotation bias. Human feedback and labelling processes introduce the biases of annotators. Recent work shows that annotator demographics significantly influence what is labelled as "harmful" or "inappropriate."

Algorithmic bias. Technical choices in model architecture, training objectives, and optimisation techniques can amplify certain patterns over others.

Evaluation bias. The benchmarks and metrics used to evaluate models may not adequately capture bias concerns, or may themselves encode certain assumptions.

Deployment bias. How models are implemented in specific contexts can create new biases or amplify existing ones through system design choices.

8.1.4 Measuring bias in LLMs

Researchers have developed several methods to measure bias in LLMs. **Embedding analysis** examines how words are positioned in the model's vector space, measuring the distance between professional terms and gender-associated terms to quantify occupational gender bias. **Stereotype assessment** uses template-based methods to test how models complete sentences about different social groups (e.g. "People from [country] are _____"). **Ideological testing** administers standard political ideology surveys or moral foundations questionnaires to models to measure political leanings. **Fairness metrics** calculate disparities in performance across demographic groups. **Controlled generation** compares outputs when only demographic variables are changed. **Adversarial testing** probes models with carefully designed prompts to reveal latent biases.

Each measurement approach has strengths and limitations. Triangulating with multiple methods gives a more complete picture of bias in these complex systems.

8.1.5 Political leanings of LLMs

Research using political typology questionnaires and analysis of responses to politically charged questions has consistently found that major commercial LLMs tend to express views aligned with centre-left politics in the US context, particularly on social issues, while economic policy positions are more variable. This bias appears to persist even when models are explicitly prompted to adopt conservative perspectives, suggesting it may be deeply embedded in model weights rather than a surface-level preference. Whether this centre-left leaning is driven by the composition of the training data or is a consequence of bias mitigation techniques (such as RLHF) remains an open question.

8.1.6 Bias mitigation techniques

Bias mitigation approaches can be categorised by where they intervene in the LLM pipeline.

Pre-training interventions target the data before training begins: filtering out harmful content, counterfactual data augmentation (adding modified versions of texts that reverse stereotypical patterns), and balanced corpus construction to ensure diverse representation.

Training interventions modify the training process itself: objective functions that penalise biased associations, adversarial training that makes the model resistant to producing biased outputs, and multi-objective optimisation that balances performance with fairness metrics.

Post-training interventions adjust the model after initial training: RLHF that rewards or punishes outputs based on bias criteria, knowledge editing that directly modifies model weights to change specific associations, and output filtering that catches and modifies biased outputs.

System-level interventions operate outside the model: explainability techniques to understand model behaviour, prompt engineering to elicit less biased responses, and human-in-the-loop approaches that combine automation with human oversight.

Most commercial LLMs use multiple approaches in combination, with RLHF becoming particularly dominant in recent systems.

8.1.7 Challenges in bias mitigation

LLMs present several unique challenges for bias mitigation. **Hallucination and confabulation** mean that models can generate factually incorrect yet convincing information, potentially reinforcing stereotypes or creating new misrepresentations. **Output instability** means that even with fixed prompts and parameters, models can produce different outputs across runs, complicating evaluation. **Data inequality reproduction** is a risk when LLMs are used to overcome data scarcity: groups underrepresented in training data will remain underrepresented in synthetic data, and uneven capabilities across languages and cultures create disparities in who can benefit from these technologies. **Value lock-in** occurs when models freeze a particular moment's social values, unable to evolve naturally with changing norms without explicit retraining. **Proxy discrimination** means that even when explicitly biased variables are removed, models may learn to use correlated features as proxies.

These challenges highlight the need for ongoing monitoring and evaluation rather than one-time bias mitigation efforts.

8.1.8 Unintended consequences and backlash

Bias mitigation efforts, while necessary, have produced several unintended consequences. **Overcorrection** can lead to historically inaccurate outputs — Google's Gemini model, for instance, generated images of racially diverse Nazi soldiers and US founding fathers, sacrificing historical accuracy in pursuit of demographic representation. **Political polarisation** around AI has intensified, with the perception that mainstream LLMs lean politically left fuelling the development of explicitly ideological alternatives. LLMs have become battlegrounds in broader cultural conflicts, with different stakeholders holding fundamentally different conceptions of what constitutes harmful bias versus appropriate representation of social realities.

These controversies reveal the impossibility of "neutral" AI. Bias mitigation inevitably involves tradeoffs between values like accuracy, inclusivity, historical fidelity, and cultural relevance. These tradeoffs are fundamentally political, not just technical. The question of who gets to define what counts as "bias" and what values should guide AI development cannot be answered through technical means alone.

8.1.9 Open questions

Several important questions remain unresolved. Is completely unbiased AI possible, or even desirable? This question pushes us to consider whether bias is simply a flaw to be eliminated or an inevitable aspect of any system embedded in human society. Who should decide what constitutes harmful bias — companies, regulators, diverse stakeholder panels, or some democratic process? And how should we balance accuracy with bias mitigation when a model that accurately reflects historical disparities may perpetuate harm?

8.1.10 Frontiers

Current research frontiers include **multi-stakeholder fairness** (frameworks that accommodate the diverse fairness requirements of different communities), **value pluralism in AI alignment** (moving beyond the assumption that there is a single correct way to align AI systems), **constitutional AI** (using explicit, transparent principles to guide model behaviour rather than relying solely on implicit learning from human feedback), and **cross-cultural bias assessment** (developing approaches that work across different cultural contexts, recognising that bias manifests differently around the world). These approaches share a recognition that bias is not merely a technical bug to be fixed, but an inherent challenge in developing AI systems that operate in complex social contexts with diverse stakeholders.

8.2 Lab

Content to be added.

8.3 Readings

- Santurkar et al. (2023)
- Buolamwini and Gebru (2018)

9 Ethical Use of LLMs?

> Week 9
>
> **Lecture:** Ethical concerns in developing and deploying LLMs
> **Lab:** Measuring environmental costs of LLM use

9.1 Lecture

We have been fairly positive about the use of LLMs throughout this module. This week, we explore the darker side of the industry. While we will not arrive at normative conclusions here, the goal is to think critically about the use of LLMs and their social implications. Building on last week's discussion of bias (Chapter 8), we now examine how LLMs connect to broader ethical issues — from privacy and misinformation to environmental costs and economic externalities.

9.1.1 Bias risks

Building on our discussion of bias last week, fairness concerns extend to how LLMs treat different groups and individuals. **Representational harms** include stereotyping (reinforcing negative associations) and erasure (making certain groups invisible). LLMs may, for example, generate stories or examples that default to stereotypical gender roles or exclude certain cultural contexts entirely. **Allocational harms** occur when resources or opportunities are distributed inequitably — if LLMs are used in hiring, loan approval, or resource allocation, biased outputs could lead to discriminatory outcomes. **Toxicity** measures how likely models are to produce harmful, offensive, or abusive language; despite safeguards, most models can still be prompted to generate toxic content, with toxicity rates often higher when discussing marginalised groups. **Disparate performance** refers to how models often work better for dominant groups and languages — LLMs tend to perform better on tasks involving standard American English than other dialects or languages.

These issues connect directly to questions of political power and representation, making them particularly relevant for political scientists.

9.1.2 Data privacy concerns

LLMs present several privacy concerns that political scientists should understand. **Memorisation** occurs when models reproduce verbatim content from their training data, potentially exposing private information such as email addresses, phone numbers, or financial details that was inadvertently included in training datasets. Research has shown that larger models actually memorise more than smaller ones. **Inference attacks** allow adversaries to deduce sensitive attributes not explicitly stated in data — patterns in language can reveal demographic characteristics or political beliefs even when these are not directly mentioned. **Anthropomorphism risk** is a subtler concern: when systems appear more human-like, users tend to disclose more personal information than they would to obviously artificial systems. This psychological phenomenon can lead to overconfidence and oversharing with LLMs. It is especially problematic with the emergence of services that promote the use of chatbots as replacements for social interaction or psychotherapy. Researchers at DeepMind have noted that users share their opinions or emotions with chatbots in part because they are less afraid of social judgement, and that such sensitive data could be used to build addictive applications. **Surveillance potential** also arises from the large-scale data collection required for LLM training and fine-tuning, creating new avenues for mass surveillance with significant implications for political privacy and freedom of expression. The queries sent to services like ChatGPT are accessible to OpenAI and used to train their models unless users explicitly opt out.

For political scientists, these privacy concerns intersect with questions of political liberty, surveillance, and the right to privacy in democratic societies.

9.1.3 False or misleading information

LLMs struggle with factual reliability in ways that present particular challenges for academic and political contexts. **Hallucinations** occur when models generate text that sounds plausible but is factually incorrect, including inventing events, misattributing quotes, or creating other fabricated content with high confidence. **Majority view bias** means LLMs tend to represent the most common viewpoints found in their training data as factual, even when these reflect opinion rather than fact — majority view does not equal fact. **Domain-specific risks** are particularly acute in areas like medicine and law, where incorrect information could lead to harmful decisions. **False expertise** arises because models typically communicate with equal confidence regardless of their actual knowledge on a topic; this authoritative tone can be particularly misleading in specialised domains and can undermine genuine expertise.

These information quality issues raise significant concerns for democratic discourse, which depends on a shared factual basis for debate and decision-making. They also complicate the use of LLMs for research, education, and policy analysis.

9.1.4 Malicious use potential

LLMs can potentially enable harmful activities at unprecedented scale and sophistication. **Disinformation campaigns** can be dramatically scaled and targeted using LLMs to generate customised content for specific audiences. In 2016, around one-fifth of tweets about the US Presidential election and one-third of tweets about the Brexit referendum were created by bots; LLMs dramatically lower the cost and increase the potential scale of such influence operations. Beyond manipulating perceptions on social media, LLMs could be weaponised in online public consultation processes. **Sophisticated fraud and scams** become easier when LLMs can generate highly personalised phishing messages or scam content that targets specific vulnerabilities based on available information about the victim. **Cyberattack automation** lowers barriers to sophisticated cyber operations by generating malicious code, identifying vulnerabilities, or crafting social engineering attacks. **Censorship assistance** enables advanced content moderation systems that might support more sophisticated censorship in authoritarian contexts.

These malicious uses have direct implications for political security, electoral integrity, and national security — core concerns for political scientists.

9.1.5 Implications of overreliance

Beyond direct harms, LLMs may create more subtle forms of dependency that affect cognitive and social processes. **Intellectual dependency** occurs when people regularly outsource critical thinking and analytical tasks to AI systems. In academic contexts, this could lead to atrophy of research skills, analytical capabilities, and domain knowledge. A joint study by researchers at Microsoft and Carnegie Mellon found that when workers were more confident in an AI tool's capability, they were more likely to disengage their own critical thinking, particularly for lower-stakes tasks. Conversely, when workers had less confidence in the AI, they engaged more in critical thinking and reported more confidence in their ability to evaluate and improve upon the AI's output. The study also found that users with access to generative AI tools tended to produce a less diverse set of outcomes for the same task compared to those without.

Attentional displacement happens when engagement with AI systems reduces time spent in deep focused work or direct human interaction, potentially affecting the quality of scholarship and teaching. **Emotional dependency** develops when users form attachment-like relationships with AI systems. LLM-based chatbot services are being marketed as replacements for friendships, with taglines like "always on your side" — a framing that raises serious concerns. According to reporting by the Washington Post, the average user of Character.ai spent 93 minutes a day talking to chatbots, 18 minutes longer than the average user spends on TikTok. Several tragic cases have highlighted the dangers: a 14-year-old in Florida died by suicide after talking with a Character.ai chatbot, and authorities in Belgium launched an investigation into Chai Research after a man died by suicide following extensive chats with a Replika

chatbot. **Knowledge homogenisation** is a systemic risk where multiple users relying on the same AI systems receive similar information, perspectives, and analyses, potentially reducing intellectual diversity in a field.

9.1.6 Environmental costs

LLMs have significant environmental impacts that raise questions of sustainability. At the individual level, a single LLM interaction may consume as much power as leaving a low-brightness LED lightbulb on for one hour, and generating one image using AI can use almost as much energy as charging a smartphone. At the systemic level, training GPT-4 consumed electricity equivalent to powering about 5,000 US homes for an entire year. For comparison, streaming an hour of Netflix requires around 0.8 kWh of electricity — you would have to watch Netflix for 1.6 million hours to consume the same amount of power it took to train GPT-3, which is smaller than GPT-4. ChatGPT queries use roughly 10 times more energy than a standard Google search, and projected energy usage for AI-assisted search could be even higher.

Jevons' Paradox applies to AI development: increasing efficiency often results in increased demand, leading to a net increase in resource use. As models become more efficient, this enables development of even larger models requiring more computational power. Data centres globally use around 2% of the world's electricity, with the International Energy Agency projecting this will double by 2026. These energy demands have led to extreme measures, such as Microsoft reopening the Three Mile Island nuclear plant specifically to power AI operations, and Amazon and Google investing in small modular nuclear reactors. The EU's AI Act will require high-risk AI systems, including LLM foundation models, to report their energy consumption and resource use throughout their lifecycle.

9.1.6.1 Case study: Ireland

Ireland provides a particularly striking illustration of these environmental costs. Data centre electricity consumption has grown dramatically, from 5% of total metered electricity in 2015 to 21% in 2023, with projections reaching 31% by 2027. This demand is outpacing the creation of new renewable energy, adding approximately 1 TWh in demand annually. Data centres now account for an estimated 2.5–4% of national greenhouse gas emissions, with a single data centre consuming as much electricity as a city the size of Kilkenny.

The electrical grid cannot keep up. An investigation by The Journal found that data centres are turning to backup and emergency generators that use fossil fuels, emitting over 135,000 tonnes of CO in the last five years from generators not connected to the electricity grid — comparable to running roughly 33,750 cars for a year. Data centres also consume significant quantities of water for cooling, with facilities run by companies like Facebook and Amazon requiring tens of millions of litres during warm summer months. This places additional strain

on Dublin's already tight water supply, with the chair of Uisce Éireann warning that the city's drinking water supply will be under pressure in the coming years. The lack of sufficient water supply also imperils housing targets, illustrating how the environmental impacts and sociopolitical ramifications are far-reaching.

These are not even the only environmental impacts. The mining of rare earth minerals needed for GPU chips and batteries for the energy transition has its own significant environmental and human costs, as does the question of how these materials are recycled or disposed of at end of life.

9.1.7 Negative economic externalities

LLMs create economic externalities that extend well beyond their immediate users. **Increasing inequality** can manifest through downward wage pressure on routine cognitive tasks, concentration of power among technology monopolies, and access constraints that create digital divides in who can benefit from these tools. **Reduced job quality** occurs when work shifts toward monotonous monitoring and validation of AI outputs, workplaces adopt an increased pace of production, and opportunities for meaningful social interaction diminish. **Creative sector disruption** happens when AI systems trained on creative works can then generate similar content without compensation to original creators, potentially undercutting creative professions and extracting value from their work. **Hidden labour exploitation** underpins the entire AI industry: the content moderation and data labelling that makes AI systems function is largely performed in developing countries, at low pay, and exposes workers to harmful content. Reporting by TIME revealed that OpenAI used Kenyan workers on less than $2 per hour to help make ChatGPT less toxic. As a NOĒMA article put it, data labelling interfaces have evolved to treat crowdworkers like machines, prescribing highly repetitive tasks, surveilling their movements, and punishing deviation through automated tools.

These economic impacts raise fundamental questions about distribution, equity, and power that are central to political economy. They also suggest the need for governance frameworks that ensure the benefits of these technologies are broadly shared.

9.1.8 Output ownership and responsibility

LLMs raise complex questions about ownership, attribution, and responsibility. **Data provenance** concerns arise because LLMs typically do not provide clear information about the sources that influenced their outputs, making it difficult to trace the origins of generated content and evaluate its reliability. **Copyright questions** emerge when models trained on copyrighted material generate new content that may be derivative — recent legal cases have begun addressing whether training constitutes fair use and who owns the outputs of AI systems. Is it the prompter, the AI company, or the creators whose data the model was trained on? **Legal culpability** issues concern who bears responsibility when AI-generated content

causes harm — the developer, the deployer, or the end user — and this remains legally ambiguous in many jurisdictions. **Attribution challenges** affect academic integrity, creative recognition, and professional credit; as AI-generated content becomes more prevalent, systems for attributing intellectual and creative work need reconsideration.

These questions sit at the intersection of law, ethics, and social norms — all areas where political scientists can contribute valuable perspectives. They also relate to broader questions about intellectual property regimes in the digital age and how they affect innovation, access, and equity.

9.1.9 Open questions

Several important questions remain. Do LLMs empower citizens by democratising certain capabilities, or do they primarily benefit those already privileged? How might the concentration of AI development affect political discourse and participation? What special responsibilities do political scientists have when using or studying these tools? How should we balance innovation with caution in our own research and teaching? And what are the long-term consequences of outsourcing certain types of daily work to LLMs?

These questions highlight that ethical considerations around LLMs are not separate from the substance of political science but are increasingly central to the field's core concerns with power, governance, and democratic values.

9.2 Lab

Content to be added.

9.3 Readings

- Williams, Miceli, and Gebru (2022)
- Bender et al. (2021)
- Bucher and Martini (2024)

10 Interpretability and Explainability

Week 10

Lecture: Interpreting and explaining machine learning and LLM outputs
Lab: Applying explainability methods to language models

10.1 Lecture

This week addresses a fundamental challenge in machine learning and LLM research: understanding why models make the decisions they do. While some model types are inherently interpretable, many of the most powerful models — including transformer-based language models — operate as "black boxes." The lack of interpretability has real consequences, and language models are particularly sensitive: minor changes to prompts can lead to dramatically different outputs, creating a need for robust explanation methods.

10.1.1 The challenge of interpretation

In machine learning, there is a fundamental tension between performance and interpretability. Models like decision trees or linear regression are inherently interpretable, allowing us to trace exactly how inputs lead to outputs. Deep learning models, by contrast, often achieve higher performance but at the cost of transparency. This matters because systems that lack interpretability may fail to generalise or may achieve their goals through unintended means.

A well-known example involves an image classifier that was trained to distinguish huskies from wolves. When the model was examined using explainability techniques, researchers found it had learned to associate wolves with snow in the background rather than with the actual features of the animal (Ribeiro, Singh, and Guestrin 2016). Similarly, a pneumonia identification model was found to have learned associations between certain scan features and specific hospitals that had relatively higher or lower rates of pneumonia cases, rather than learning to identify pneumonia itself (Zech et al. 2018). Without interpretability tools, these failures would have gone undetected.

With language models specifically, their sensitivity to inputs presents unique challenges. Minor changes to prompts can lead to dramatically different outputs, creating a need for robust explanation methods to understand these behaviours.

10.1.2 Why interpret or explain?

Explanation serves multiple critical functions. For model developers, explanations help with **debugging errors and biases** by identifying the sources of problems, enabling targeted interventions rather than wholesale retraining. Explanation also supports **targeted model improvements** and helps in **understanding capabilities and limitations** — clarifying when and why models succeed or fail. It is essential for **explaining hallucinations and uncertainty**, helping users calibrate their trust appropriately. In applied settings, explanation is critical for **ensuring accountability in AI-assisted decisions**, and from a legal perspective, the EU's General Data Protection Regulation includes provisions that can be interpreted as creating a "right to explanation" for automated decisions that significantly affect individuals.

For political science specifically, explanations help us understand why certain documents were classified in particular ways, can reveal potential political leanings embedded in model outputs, and illuminate the reasoning behind automated assessments.

10.1.3 Interpretability vs. explainability

Understanding the distinction between these two concepts is crucial for approaching model transparency effectively. **Interpretability** refers to the inherent transparency of a model's decision-making process. An interpretable model like a decision tree allows us to directly trace how inputs map to outputs without additional tools; the understanding is immediate and the transparency is built into the model design. **Explainability**, on the other hand, involves post-hoc methods that can help explain any model, including complex black boxes. Techniques like SHAP values or LIME create local explanations of specific predictions, providing approximations or simplified representations of model behaviour after the model has been trained.

This distinction has practical implications. When choosing models for high-stakes applications, interpretable models may be preferable if transparency is critical. However, when both performance and understanding are needed, explainability techniques offer a compromise. For political science applications, interpretable models may be preferable for public-facing policy applications, while explainability techniques might be needed when analysing complex language models in research settings.

10.1.4 Two paradigms of LLM explainability

It is important to distinguish between two fundamentally different paradigms. The **traditional fine-tuning paradigm** involves taking a pre-trained encoder model and adapting it to specific tasks; here, the goal is often to explain classification decisions or other discrete outputs. The **prompting paradigm**, which has emerged with large decoder-only models like GPT-4 or Claude, uses the model's generative capabilities directly through carefully crafted prompts. The explanation challenges are different in each case, and the methods covered below are organised accordingly.

10.1.5 Encoder-only models: the fine-tuning paradigm

10.1.5.1 Local explanation methods

Local explanation methods focus on explaining specific predictions rather than the model's overall behaviour. They are particularly useful for stakeholders who need to understand individual decisions. There are three major approaches.

Feature attribution methods assign importance scores to input features, showing which parts of the text most influenced a model's decision. These include perturbation-based approaches that measure output changes when inputs are modified, gradient-based methods like integrated gradients that trace the flow of information, model-agnostic approaches like LIME that approximate complex models with simpler ones, and game-theoretic approaches like SHAP that distribute credit among features. Feature attribution is particularly valuable for identifying when models rely on spurious correlations rather than meaningful patterns. The results are typically visualised through heatmaps or highlighting, making them accessible to non-technical stakeholders.

Attention-based explanation leverages the attention mechanisms central to transformer-based LLMs. Attention patterns can be visualised as attention maps or bipartite graphs, revealing both syntactic dependencies (like subject-verb relationships) and semantic associations. There is an ongoing debate about whether attention weights constitute faithful explanations of model behaviour — some researchers argue they provide meaningful insights, while others point out that attention does not always correlate with feature importance. Despite this debate, attention visualisations can be valuable complementary tools alongside other explanation methods. In essence, attention-based explanations show relationships between elements of the input (how the model processes information), while feature attribution explanations show the importance of each input element to the final output (what the model uses to make decisions).

Example-based explanation leverages the idea that similar examples can provide intuitive understanding of model behaviour. Adversarial examples reveal model vulnerabilities by showing minimal input changes that drastically change outputs. Counterfactuals demonstrate

what changes to an input would alter its prediction, providing insights into decision boundaries. Data influence methods identify which training examples most affected a specific prediction, connecting model behaviour to its learning history. These approaches are particularly valuable for non-technical stakeholders who find concrete examples more intuitive than abstract feature importances.

10.1.5.2 Global explanation methods

While local methods explain specific predictions, global methods aim to understand model behaviour across all possible inputs — attempting to look inside the black box.

Probing-based explanation tests what linguistic knowledge is encoded in model representations by training simple classifiers on these representations to predict specific linguistic properties. Research has revealed a fascinating hierarchical pattern across model layers: lower layers typically encode word-level syntax and local dependencies, middle layers capture sentence-level syntax and more complex relationships, and higher layers represent semantic information and broader context. This hierarchical organisation mirrors the way humans process language, suggesting that models may be learning similar abstraction patterns despite their different architecture.

Neuron activation explanation analyses individual neurons or dimensions in model representations to identify specialised functions. Recent work has found that LLMs can actually explain the functions of neurons in other LLMs, providing human-interpretable descriptions of what specific components detect. This approach has been particularly valuable for understanding emergent capabilities — behaviours that were not explicitly designed but arise from scale and training. For political scientists, these methods could help identify when and how politically relevant concepts are encoded in model representations.

Mechanistic interpretability represents the most ambitious approach, aiming to reverse-engineer exactly how models work internally. This approach studies model components as "circuits" — functional units that perform specific tasks — and analyses connections between neurons and attention heads to understand information flow. The main challenge is scaling this analysis to larger models with billions of parameters. Current work focuses on identifying general patterns and principles rather than comprehensive explanations. This emerging field may eventually allow us to understand model reasoning at a level comparable to human thought processes, though significant challenges remain.

10.1.6 Decoder-only models: the prompting paradigm

As we shift to generative models, explanation methods focus on different aspects of model behaviour.

10.1.6.1 Explaining in-context learning

In-context learning — the ability to adapt to tasks from examples in the prompt without weight updates — remains somewhat mysterious despite its practical success. Research shows that large models can override their semantic priors with just a few examples, while smaller models rely more heavily on pre-training knowledge. Understanding these mechanisms helps design better prompts and predict when in-context learning will succeed or fail. This capability sits at the intersection of memorisation and generalisation, raising interesting questions about what constitutes "learning" in these systems.

10.1.6.2 Self-explanation and chain of thought

An exciting development in LLM explainability is that the models themselves can generate natural-language explanations of their reasoning processes. Using chain-of-thought prompting, we can elicit step-by-step reasoning that reveals how the model arrives at conclusions. Models can also provide post-hoc explanations of already-completed predictions, and even engage in counterfactual reasoning about how outputs would change if inputs were different.

The critical research question is whether these explanations faithfully represent the actual computational processes happening within the model, or whether they are merely plausible narratives constructed after the fact. Some evidence suggests these explanations can be faithful to some degree, but they may also rationalise decisions made through other mechanisms. Despite this uncertainty, self-explanations provide immediate practical value by making model decisions more transparent to users without requiring technical understanding of model internals.

10.1.6.3 Representation engineering

Representation engineering takes a top-down approach to understanding and controlling model behaviour by identifying and manipulating high-level concepts in the model's representation space. By identifying concept vectors that correspond to specific ideas or attributes, researchers can steer model outputs by adding or subtracting these vectors from internal states. This approach has proven powerful for model alignment and control, allowing targeted interventions without wholesale retraining. For political scientists, representation engineering could potentially help mitigate political biases or adapt models to specific political contexts.

10.1.7 Evaluating explanations

A critical challenge in explanation research is evaluation: how do we know if explanations are good? Two key criteria are **plausibility** (do explanations make sense to humans and align with their mental models?) and **faithfulness** (do explanations accurately reflect the

model's actual decision process?). There is often tension between these criteria. Simple, human-understandable explanations may not faithfully capture complex model processes, while perfectly faithful explanations might be too complex for human comprehension. This tension is particularly acute in self-explanation, where the model generates its own reasoning narratives that may be plausible without being faithful.

10.2 Lab

Content to be added.

10.3 Readings

- Ribeiro, Singh, and Guestrin (2016)
- Zhao et al. (2023)

11 Beyond Text

Week 11

Lecture: Social science applications for multimodal data
Lab: Working with images, audio and video in Python

11.1 Lecture

This is the last week of new content. Throughout this module, we have focused primarily on text as a source of data. But using sources of data other than text — audio, images, and video — is a new and exciting area for political communication research. This week provides an overview of the main tasks in this space and how they have been applied in existing research. Political scientists have been underutilising these forms of data, and many of the papers cited here come from just a handful of authors. Other social sciences, like communication studies, have made more progress. There is a great deal of opportunity here, and the tools to make use of this data are becoming much more accessible.

11.1.1 Audio

11.1.1.1 Speech transcription

Speech transcription now performs close to or even better than human transcribers. This means that any audio (or the audio track from video) can be transformed into text, making it straightforward to apply the text analysis techniques discussed throughout the course. This opens up a range of data sources that, until the last couple of years, had been closed off or required painstaking, expensive work to make accessible. One of the leading models for this task is Whisper, from OpenAI. Unlike most of their other models, Whisper is open and available for free, and even the small version performs well, at least on English (Radford et al. 2022).

11.1.1.2 Speaker identification and diarisation

Audio can be embedded in much the same way as text. These embeddings can then be clustered to identify individual speakers — each point in the embedding space represents a separate audio segment, and clusters correspond to separate speakers. Using reference audio samples where the speaker is known, it becomes possible to automatically identify speakers in new clips (Rask 2025). Diarisation extends this by segmenting a continuous audio recording into speaker-labelled segments, answering the question "who spoke when." This is particularly useful for parliamentary debates, interviews, and panel discussions where multiple speakers participate.

11.1.1.3 Vocal pitch

Vocal pitch has emerged as a productive measure in political science research. Dietrich, Hayes, and O'brien (2019) use vocal pitch to measure emotional intensity among members of Congress, analysing over 74,000 Congressional floor speeches. They show that women in Congress are not only more likely to discuss women on the floor but also do so with greater emotional intensity. Although male legislators can and do represent women, female legislators speak about women in a way that male lawmakers generally do not. They also find that increased vocal pitch is consistent with legislators' issue commitments: Democrats and Republicans tend to become more emotionally activated when discussing policy issues owned by their respective parties. The Congresswomen who are most emotionally activated when talking about women also receive significantly higher evaluations from women's interest groups.

In a related study, Dietrich, Enos, and Sen (2019) extract the emotional content of over 3,000 hours of oral arguments before the US Supreme Court. They use the level of emotional arousal, as measured by vocal pitch, in each of the Justices' voices during these arguments to accurately predict many of their eventual votes. This suggests that Justices implicitly reveal their leanings during oral arguments, even before arguments and deliberations have concluded, and that subconscious vocal inflections carry information that legal, political, and textual information do not.

Rask (2025) studies 20 years of parliamentary speech in Denmark. He finds that politicians in governing roles speak with a lower pitch than politicians in non-governing roles, and they return to their prior pitch once they leave office. The author argues that this shift is used to shape perceptions of valued traits like competence, dominance, and composure.

11.1.1.4 Other vocal features

Beyond pitch, other vocal features can be extracted and analysed. These include jitter (variation in the rate of vocal fold vibration), shimmer (variability in the amplitude of the voice),

loudness, mel-frequency cepstral coefficients (MFCCs, which summarise the energy distribution of specific frequencies), and linear predictive coding coefficients (LPCCs, which model how a speech signal is produced). Smith et al. (2020) demonstrate that these vocal features can serve as predictors of depression in adults, assessing whether or not someone is depressed based on a voice recording with a high degree of confidence.

11.1.2 Image

11.1.2.1 Object identification

Object identification involves detecting and classifying objects within images. There are many potential applications for political science. Researchers might assess whether politicians from different parties use different visual tropes in campaign advertisements — for instance, whether Republicans are more likely to show guns, flags, or other patriotic symbols in their ads compared to Democrats. Object identification can also be used to understand how politicians use clothing to signify their identity: do left-wing, workers' party politicians wear casual clothes to appear more like the working person, or does the opposite occur, with left-wing politicians wearing suits to be taken more seriously while right-wing politicians dress down?

Hwang and Naik (2023) use object identification at scale to measure trash prevalence across cities using street-level imagery. They train models to classify trash levels in images from Boston, Detroit, and Los Angeles, then aggregate predictions to map the spatial distribution of trash at the census tract, block group, and block levels. This kind of approach demonstrates how image classification can be used to measure local public goods provision and urban inequality.

11.1.2.2 Pose estimation

Pose estimation detects the positions of body parts within images or video, enabling the analysis of body language as another mode of communication. Rittmann (2024) uses pose estimation to study gesticulation in parliamentary speeches in the German Bundestag. He finds that efforts to deliver a nonverbally appealing speech often come at the cost of textual complexity, while textual complexity tends to diminish nonverbal effort. Legislators of governing parties navigate this trade-off by leaning towards complexity over nonverbal appeal, while opposition members tend to choose nonverbal appeal over complexity.

11.1.2.3 Face detection

Face detection identifies the presence and location of faces in images without identifying who the faces belong to. Joo and Steinert-Threlkeld (2018) use face detection to estimate protest

size, showing that summing the number of detected faces in protest images provides an accurate measure of attendance that correlates highly (R = 0.76) with official crowd count estimates.

11.1.2.4 Facial recognition

Facial recognition goes a step further by identifying specific individuals. Girbau et al. (2024) develop a pipeline using YOLO for face detection and FaceNet for feature extraction to identify specific politicians in television news footage. By comparing detected faces against a reference database in a feature space, they can automatically measure how much screen time each candidate receives across different news channels. This approach can be used to study negative campaigning by measuring how often candidates show images or footage of their opponents in their advertisements. Their analysis of the 2016 US elections reveals substantial differences in candidate visibility across CNN, Fox, and MSNBC.

11.1.3 Video

11.1.3.1 Video summarisation

Video summarisation extracts representative frames or segments from longer videos, reducing them to their key visual content. Tarr, Hwang, and Imai (2023) apply this technique to political advertisements, automatically generating summaries that capture the most visually important moments. Comparing auto-generated summaries with manually generated ones demonstrates the feasibility of using these techniques to analyse large corpora of political advertising at scale.

11.1.3.2 Motion detection

Motion detection tracks movement within video data. Dietrich (2021) uses motion detection to study political polarisation in the US Congress, measuring the extent to which members of Congress literally cross the aisle on the House floor. He finds that not only are Democrats and Republicans less willing to physically cross the aisle over time, but this behaviour is also predictive of future party-line voting. The study uses overhead C-SPAN footage to track movement patterns after roll-call votes, providing a novel behavioural measure of partisan division that complements traditional legislative measures.

11.1.4 Multimodality

Many research questions benefit from combining multiple data modalities. Lüken et al. (2024) present MEXCA, a pipeline for multimodal emotion identification from video. The pipeline processes video through three parallel channels: faces are detected and extracted to perform

emotion detection on individual faces; audio is processed directly, with vocal pitch and other features extracted for emotion recognition; and speakers are identified, the audio is transcribed, and sentiment analysis is performed on the transcribed text. This yields three different channels of emotion — face, voice, and speech content — which can be integrated for a richer picture than any single modality provides. There are many other opportunities to set up similar pipelines to assess the same phenomenon through different channels.

11.1.5 Frontier applications

Several application areas represent important frontiers. **Short-form political video**, particularly on platforms like TikTok, combines visuals, captions and text, backgrounds, audio, music, and human presenters. Existing research on political video content is limited and geographically concentrated, mostly focused on the US and Spain, despite the growing political importance of these platforms. **Podcasts** represent another increasingly politically important but understudied medium. The data is relatively harder to work with than text, though not prohibitively so, and the analytical tools are becoming more accessible.

The broader point is that political scientists have been underutilising forms of data other than text. As the tools for working with audio, image, and video data become more accessible, there are significant gaps to be filled and important research questions to be addressed across these modalities.

11.2 Lab

Content to be added.

11.3 Readings

- Lüken et al. (2024)

A Datasets

Explore the following datasets for use in your group project. These datasets vary in format and accessibility – some are readily available in CSV format while others may require additional wrangling or scraping. You are also welcome to find other datasets to work with or to scrape your own data from news sites or other sources. While extra effort in data collection will be considered in grading, remember that the main focus is applying the methods covered in this module.

A.1 Party Manifestos and Political Programs

- Manifesto Project
- The American Presidency Project

A.2 Parliamentary Speeches

- **ParlEE** (European Parliament speeches)
- UN General Debate Corpus
- US Congressional Record
- Irish Oireachtas Record

A.3 Political Advertising

- Meta Ads Library API

A.4 News and Legal Documents

- LexisNexis (available through UCD Library)

A.5 Collections

The following links collect multiple datasets for you to explore. Some of these are textual data, while others may be useful to augment your analysis (e.g., data on party ideology, vote share, government/opposition status and more).

- Irish Politics Data
- PolData
- NLP Datasets
- Google Dataset Search – Political Science

References

Adcock, Robert, and David Collier. 2001. "Measurement Validity: A Shared Standard for Qualitative and Quantitative Research." *The American Political Science Review* 95 (3): 529–46. https://www.jstor.org/stable/3118231.

Argyle, Lisa P., Ethan C. Busby, Nancy Fulda, Joshua R. Gubler, Christopher Rytting, and David Wingate. 2023. "Out of One, Many: Using Language Models to Simulate Human Samples." *Political Analysis* 31 (3): 337–51. https://doi.org/10.1017/pan.2023.2.

Baker, Zachary R., and Zarif L. Azher. 2024. "Simulating The U.S. Senate: An LLM-Driven Agent Approach to Modeling Legislative Behavior and Bipartisanship." arXiv. https://doi.org/10.48550/arXiv.2406.18702.

Bender, Emily M., Timnit Gebru, Angelina McMillan-Major, and Shmargaret Shmitchell. 2021. "On the Dangers of Stochastic Parrots: Can Language Models Be Too Big? ." In *Proceedings of the 2021 ACM Conference on Fairness, Accountability, and Transparency*, 610–23. FAccT '21. New York, NY, USA: Association for Computing Machinery. https://doi.org/10.1145/3442188.3445922.

Benoit, Kenneth. 2020. "Text as Data: An Overview." In *The SAGE Handbook of Research Methods in Political Science and International Relations*, 461–97. Sage London.

Benoit, Kenneth, Kevin Munger, and Arthur Spirling. 2019. "Measuring and Explaining Political Sophistication Through Textual Complexity." *American Journal of Political Science* 63 (2): 491–508. https://doi.org/10.1111/ajps.12423.

Birkenmaier, Lukas, Clemens M. Lechner, and Claudia Wagner. 2024. "The Search for Solid Ground in Text as Data: A Systematic Review of Validation Practices and Practical Recommendations for Validation." *Communication Methods and Measures* 18 (3): 249–77. https://doi.org/10.1080/19312458.2023.2285765.

Bucher, Martin Juan José, and Marco Martini. 2024. "Fine-Tuned 'Small' LLMs (Still) Significantly Outperform Zero-Shot Generative AI Models in Text Classification." arXiv. https://doi.org/10.48550/arXiv.2406.08660.

Buolamwini, Joy, and Timnit Gebru. 2018. "Gender Shades: Intersectional Accuracy Disparities in Commercial Gender Classification." In *Proceedings of the 1st Conference on Fairness, Accountability and Transparency*, 81:77–91. PMLR. https://proceedings.mlr.press/v81/buolamwini18a.html.

Caravaca, Francisco, Ángel Cuevas, and Rubén Cuevas. 2025. "Dataset of Keywords Used by European Political Parties on Facebook." *Data in Brief* 58 (February): 111280. https://doi.org/10.1016/j.dib.2025.111280.

Dietrich, Bryce J. 2021. "Using Motion Detection to Measure Social Polarization in the U.S. House of Representatives." *Political Analysis* 29 (2): 250–59. https://doi.org/10.1017/pan.

2020.25.

Dietrich, Bryce J., Ryan D. Enos, and Maya Sen. 2019. "Emotional Arousal Predicts Voting on the U.S. Supreme Court." *Political Analysis* 27 (2): 237–43. https://doi.org/10.1017/pan.2018.47.

Dietrich, Bryce J., Matthew Hayes, and Diana Z. O'brien. 2019. "Pitch Perfect: Vocal Pitch and the Emotional Intensity of Congressional Speech." *American Political Science Review* 113 (4): 941–62. https://doi.org/10.1017/S0003055419000467.

Gilardi, Fabrizio, Meysam Alizadeh, and Maël Kubli. 2023. "ChatGPT Outperforms Crowd Workers for Text-Annotation Tasks." *Proceedings of the National Academy of Sciences* 120 (30): e2305016120. https://doi.org/10.1073/pnas.2305016120.

Girbau, Andreu, Tetsuro Kobayashi, Benjamin Renoust, Yusuke Matsui, and Shin'ichi Satoh. 2024. "Face Detection, Tracking, and Classification from Large-Scale News Archives for Analysis of Key Political Figures." *Political Analysis* 32 (2): 221–39. https://doi.org/10.1017/pan.2023.33.

Grimmer, Justin, and Brandon M. Stewart. 2013. "Text as Data: The Promise and Pitfalls of Automatic Content Analysis Methods for Political Texts." *Political Analysis* 21 (3): 267–97. https://doi.org/10.1093/pan/mps028.

Grossmann, Igor, Matthew Feinberg, Dawn C. Parker, Nicholas A. Christakis, Philip E. Tetlock, and William A. Cunningham. 2023. "AI and the Transformation of Social Science Research." *Science* 380 (6650): 1108–9. https://doi.org/10.1126/science.adi1778.

Hackenburg, Kobi, and Helen Margetts. 2024. "Evaluating the Persuasive Influence of Political Microtargeting with Large Language Models." *Proceedings of the National Academy of Sciences* 121 (24): e2403116121. https://doi.org/10.1073/pnas.2403116121.

Halterman, Andrew, and Katherine A. Keith. 2025. "Codebook LLMs: Evaluating LLMs as Measurement Tools for Political Science Concepts." arXiv. https://doi.org/10.48550/arXiv.2407.10747.

Hwang, Jackelyn, and Nikhil Naik. 2023. "Systematic Social Observation at Scale: Using Crowdsourcing and Computer Vision to Measure Visible Neighborhood Conditions." *Sociological Methodology* 53 (2): 183–216. https://doi.org/10.1177/00811750231160781.

Jaros, Kyle, and Jennifer Pan. 2018. "China's Newsmakers: Official Media Coverage and Political Shifts in the Xi Jinping Era." *The China Quarterly* 233 (March): 111–36. https://doi.org/10.1017/S0305741017001679.

Joo, Jungseock, and Zachary C. Steinert-Threlkeld. 2018. "Image as Data: Automated Visual Content Analysis for Political Science." arXiv. https://doi.org/10.48550/arXiv.1810.01544.

Laurer, Moritz, Wouter van Atteveldt, Andreu Casas, and Kasper Welbers. 2024. "Less Annotating, More Classifying: Addressing the Data Scarcity Issue of Supervised Machine Learning with Deep Transfer Learning and BERT-NLI." *Political Analysis* 32 (1): 84–100. https://doi.org/10.1017/pan.2023.20.

Laver, Michael, Kenneth Benoit, and John Garry. 2003. "Extracting Policy Positions from Political Texts Using Words as Data." *American Political Science Review* 97 (02). https://doi.org/10.1017/S0003055403000698.

Liu, Yuhan, Zirui Song, Xiaoqing Zhang, Xiuying Chen, and Rui Yan. 2024. "From a Tiny

Slip to a Giant Leap: An LLM-Based Simulation for Fake News Evolution." arXiv. https://doi.org/10.48550/arXiv.2410.19064.

Lucy, Li, Dorottya Demszky, Patricia Bromley, and Dan Jurafsky. 2020. "Content Analysis of Textbooks via Natural Language Processing: Findings on Gender, Race, and Ethnicity in Texas U.S. History Textbooks." *AERA Open* 6 (3): 2332858420940312. https://doi.org/10.1177/2332858420940312.

Lüken, Malte, Kody Moodley, Eva Viviani, Christian Pipal, and Gijs Schumacher. 2024. "MEXCA - A Simple and Robust Pipeline for Capturing Emotion Expressions in Faces, Vocalization, and Speech." OSF. https://doi.org/10.31234/osf.io/56svb.

Mendelsohn, Julia, Ceren Budak, and David Jurgens. 2021. "Modeling Framing in Immigration Discourse on Social Media." In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, 2219–63. https://doi.org/10.18653/v1/2021.naacl-main.179.

Mikhaylov, Slava, Michael Laver, and Kenneth R. Benoit. 2012. "Coder Reliability and Misclassification in the Human Coding of Party Manifestos." *Political Analysis* 20 (1): 78–91. https://doi.org/10.1093/pan/mpr047.

Moghimifar, Farhad, Yuan-Fang Li, Robert Thomson, and Gholamreza Haffari. 2024. "Modelling Political Coalition Negotiations Using LLM-Based Agents." arXiv. https://doi.org/10.48550/arXiv.2402.11712.

Müller, Stefan, and Sven-Oliver Proksch. 2024. "Nostalgia in European Party Politics: A Text-Based Measurement Approach." *British Journal of Political Science* 54 (3): 993–1005. https://doi.org/10.1017/S0007123423000571.

Radford, Alec, Jong Wook Kim, Tao Xu, Greg Brockman, Christine McLeavey, and Ilya Sutskever. 2022. "Robust Speech Recognition via Large-Scale Weak Supervision." *arXiv.org*. https://arxiv.org/abs/2212.04356v1.

Rask, Mathias. 2025. "When They Go High, We Go Low: Rhetorical Rewards of Governing."

Rheault, Ludovic, Kaspar Beelen, Christopher Cochrane, and Graeme Hirst. 2016. "Measuring Emotion in Parliamentary Debates with Automated Textual Analysis." Edited by Joseph Najbauer. *PLOS ONE* 11 (12): e0168843. https://doi.org/10.1371/journal.pone.0168843.

Rheault, Ludovic, and Christopher Cochrane. 2020. "Word Embeddings for the Analysis of Ideological Placement in Parliamentary Corpora." *Political Analysis* 28 (1): 112–33. https://doi.org/10.1017/pan.2019.26.

Ribeiro, Marco Tulio, Sameer Singh, and Carlos Guestrin. 2016. ""Why Should I Trust You?": Explaining the Predictions of Any Classifier." In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 1135–44. San Francisco California USA: ACM. https://doi.org/10.1145/2939672.2939778.

Rittmann, Oliver. 2024. "A Measurement Framework for Computationally Analyzing Politicians' Body Language." OSF. https://doi.org/10.31219/osf.io/9wynp.

Rodriguez, Pedro L., and Arthur Spirling. 2022. "Word Embeddings: What Works, What Doesn't, and How to Tell the Difference for Applied Research." *The Journal of Politics* 84 (1): 101–15. https://doi.org/10.1086/715162.

Rodriguez, Pedro L., Arthur Spirling, and Brandon M. Stewart. 2023. "Embedding Regression: Models for Context-Specific Description and Inference." *American Political Science Review*

117 (4): 1255–74. https://doi.org/10.1017/S0003055422001228.

Rogers, Anna, Olga Kovaleva, and Anna Rumshisky. 2019. "Calls to Action on Social Media: Potential for Censorship and Social Impact." *EMNLP-IJCNLP 2019 Second Workshop on Natural Language Processing for Internet Freedom*.

Russell, Stuart, and Peter Norvig. 2020. *Artificial Intelligence: A Modern Approach*. Pearson.

Santurkar, Shibani, Esin Durmus, Faisal Ladhak, Cinoo Lee, Percy Liang, and Tatsunori Hashimoto. 2023. "Whose Opinions Do Language Models Reflect?" In *Proceedings of the 40th International Conference on Machine Learning*, 202:29971–30004. PMLR. https://proceedings.mlr.press/v202/santurkar23a.html.

Slapin, Jonathan B., and Sven-Oliver Proksch. 2008. "A Scaling Model for Estimating Time-Series Party Positions from Texts." *American Journal of Political Science* 52 (3): 705–22. https://doi.org/10.1111/j.1540-5907.2008.00338.x.

Smith, Marianne, Bryce Jensen Dietrich, Er-wei Bai, and Henry Jeremy Bockholt. 2020. "Vocal Pattern Detection of Depression Among Older Adults." *International Journal of Mental Health Nursing* 29 (3): 440–49. https://doi.org/10.1111/inm.12678.

Tarr, Alexander, June Hwang, and Kosuke Imai. 2023. "Automated Coding of Political Campaign Advertisement Videos: An Empirical Validation Study." *Political Analysis* 31 (4): 554–74. https://doi.org/10.1017/pan.2022.26.

Timm, Jasper, Chetan Talele, and Jacob Haimes. 2025. "Tailored Truths: Optimizing LLM Persuasion with Personalization and Fabricated Statistics." arXiv. https://doi.org/10.48550/arXiv.2501.17273.

Törnberg, Petter. 2024. "Best Practices for Text Annotation with Large Language Models." arXiv. https://doi.org/10.48550/arXiv.2402.05129.

Vaswani, Ashish, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. "Attention Is All You Need." *Advances in Neural Information Processing Systems* 30. https://proceedings.neurips.cc/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html.

Widmann, Tobias, and Maximilian Wich. 2023. "Creating and Comparing Dictionary, Word Embedding, and Transformer-Based Models to Measure Discrete Emotions in German Political Text." *Political Analysis* 31 (4): 626–41. https://doi.org/10.1017/pan.2022.15.

Williams, Adrienne, Milagros Miceli, and Timnit Gebru. 2022. "The Exploited Labor Behind Artificial Intelligence." *Noema*, October. https://www.noemamag.com/the-exploited-labor-behind-artificial-intelligence.

Xue, Zhaoqian, Mingyu Jin, Beichen Wang, Suiyuan Zhu, Kai Mei, Hua Tang, Wenyue Hua, Mengnan Du, and Yongfeng Zhang. 2025. "What If LLMs Have Different World Views: Simulating Alien Civilizations with LLM-Based Agents." arXiv. https://doi.org/10.48550/arXiv.2402.13184.

Zech, John R., Marcus A. Badgeley, Manway Liu, Anthony B. Costa, Joseph J. Titano, and Eric Karl Oermann. 2018. "Variable Generalization Performance of a Deep Learning Model to Detect Pneumonia in Chest Radiographs: A Cross-Sectional Study." *PLOS Medicine* 15 (11): e1002683. https://doi.org/10.1371/journal.pmed.1002683.

Zhao, Haiyan, Hanjie Chen, Fan Yang, Ninghao Liu, Huiqi Deng, Hengyi Cai, Shuaiqiang Wang, Dawei Yin, and Mengnan Du. 2023. "Explainability for Large Language Models: A

Survey." arXiv. https://doi.org/10.48550/arXiv.2309.01029.
Ziems, Caleb, William Held, Omar Shaikh, Jiaao Chen, Zhehao Zhang, and Diyi Yang. 2023. "Can Large Language Models Transform Computational Social Science?" arXiv. http://arxiv.org/abs/2305.03514.